03

Módulo 03 - Transformers

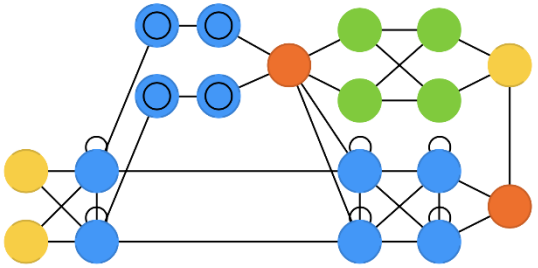
"The true art of memory is the art of attention!"
— Samuel Johnson, 1787

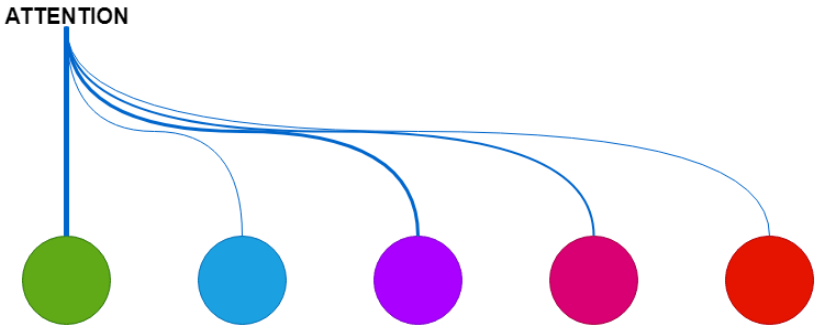
Inteligência Artificial Generativa e Modelos de Linguagem de Grande Escala (LLMs)

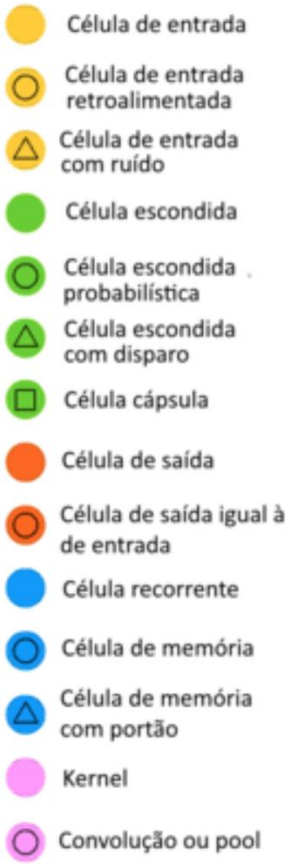
Resumo do Módulo

- Origem e evolução dos Transformers.
- Arquitetura básica dos Transformers.
- Self-attention e suas vantagens.
- Aplicações em tarefas de processamento de linguagem natural (PLN).
- Lab

Attention networks (AN) - rede de atenção







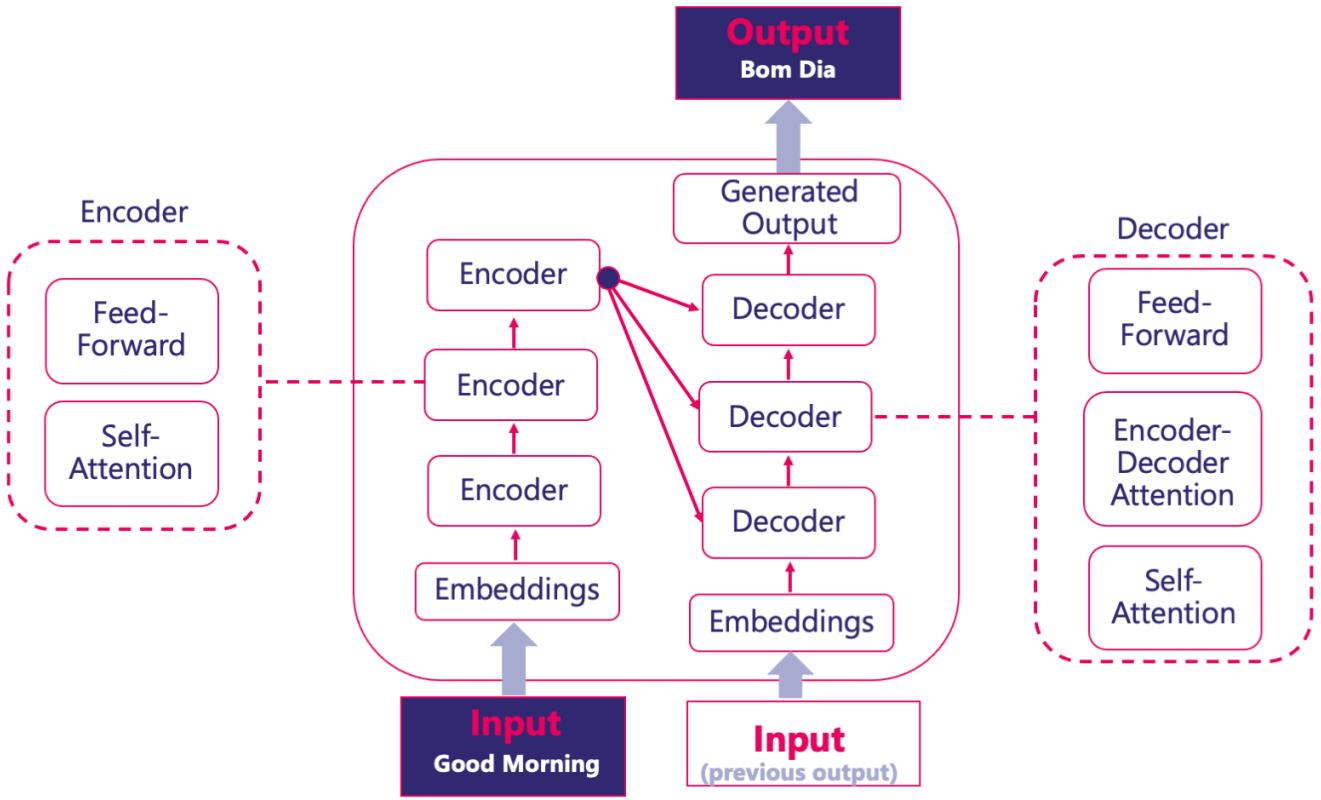
- Usa a arquitetura chamada de TRANSFORMER.
  - Usa uma memória acedida pelo mecanismo de atenção, que define a importância de cada informação a cada passo
  - Pode substituir RNNs e redes derivadas, eliminando o componente temporal do processamento
  - Permite paralelização, resultando em treino mais rápido

(Ampla aplicação em processamento de linguagem natural - NLP

Exemplo: o modelo BERT, que usa essa estrutura para produzir codificações semânticas, permitindo tarefas como classificação de documentos, resposta a perguntas, e identificação de parafraseamento e sarcasmo)

Origem e evolução dos Transformers.

- **Transformers** é a nova arquitetura de rede neural simples, introduzida pelo **Google Brain em 2017** com seu famoso trabalho de pesquisa **"Attention is all you need"**.

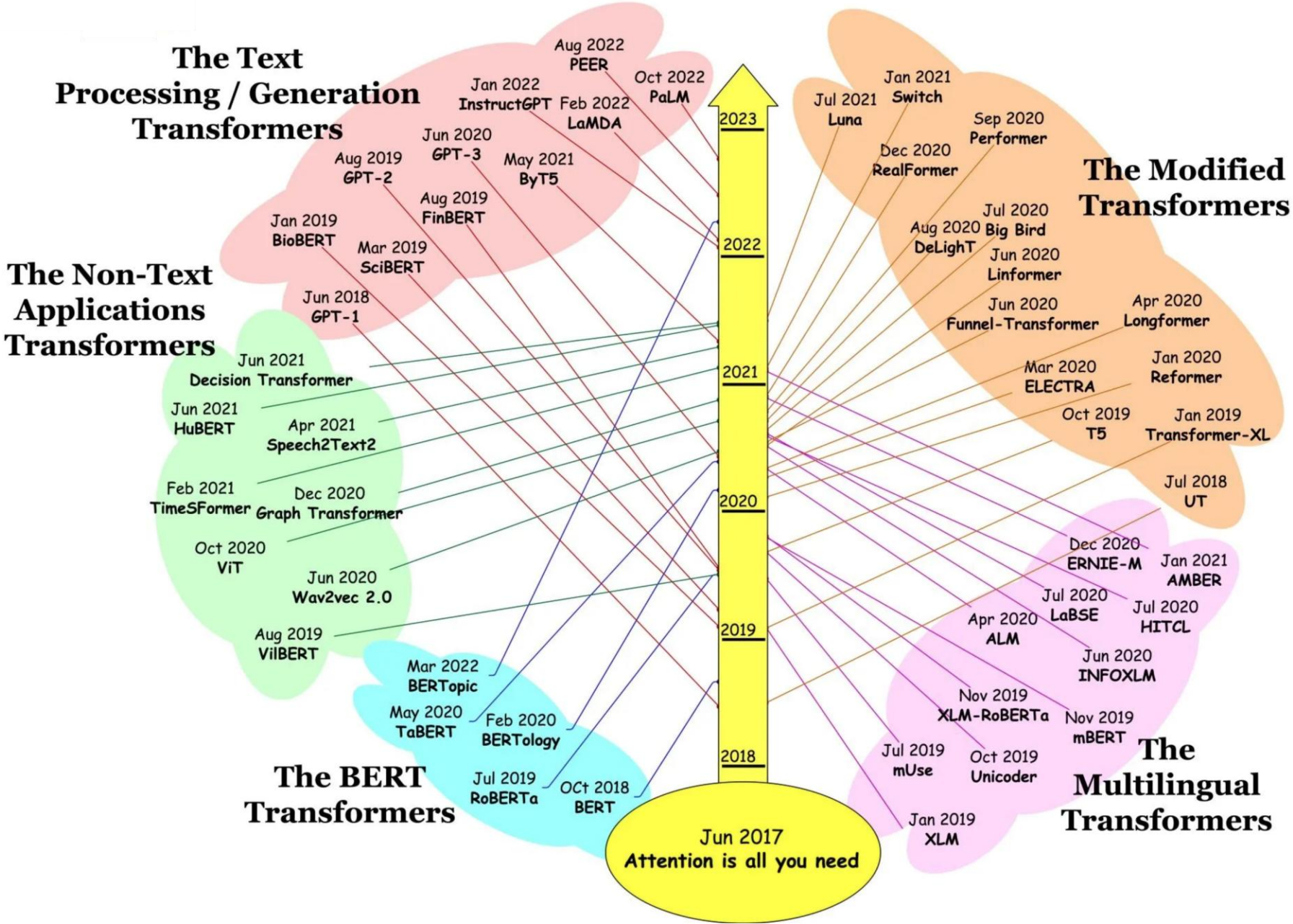


Transformers representam mais do que apenas um avanço técnico na IA!

→ simbolizam uma **mudança de paradigma** na forma como abordamos problemas complexos de processamento de informações

https://arxiv.org/abs/1706.03762

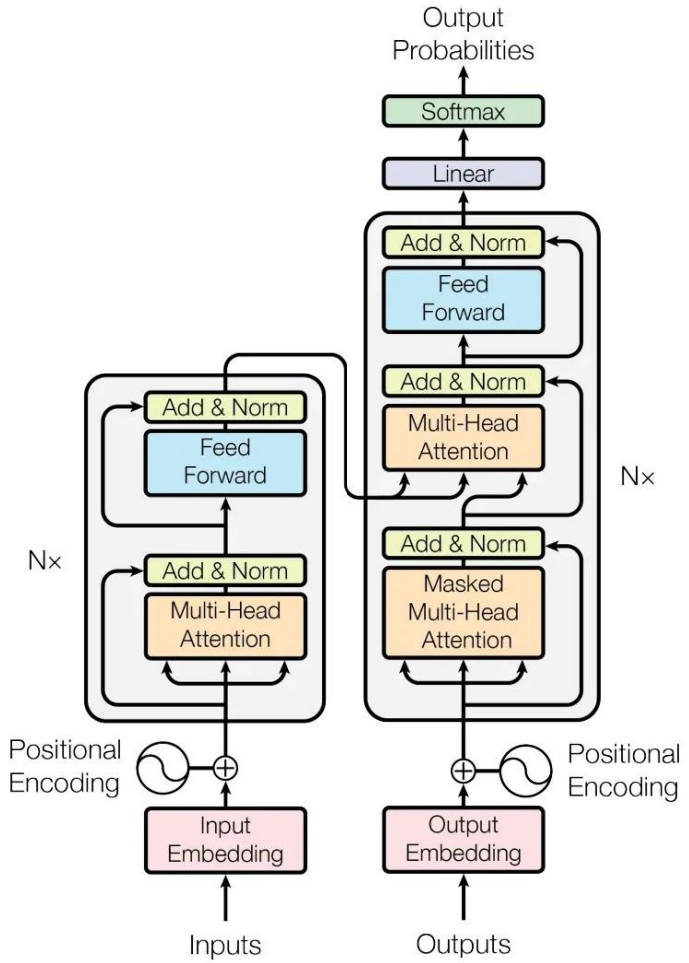
Muitos Transformers desde então



0 Transformer encoder-decoder

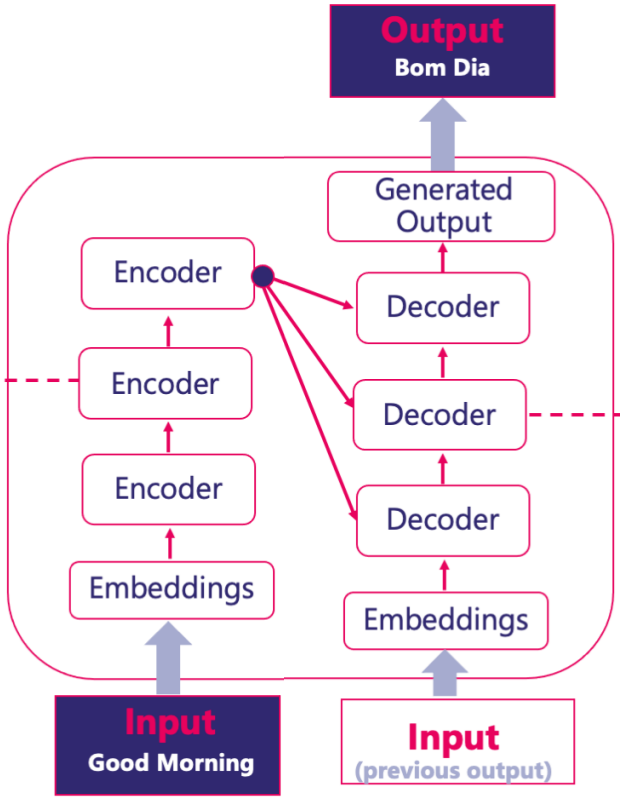
BERT

Encoder



GPT

Decoder



BERT (Bidirectional Encoder Representations from Transformers)

GPT (Generative Pretrained Transformer)

Transformers 2.0? (a 31 de Dezembro de 2024?)

Titans: Learning to Memorize at Test Time

Ali Behrouz[†], Peilin Zhong[†], and Vahab Mirrokni[†]

[†]Google Research

{alibehrouz, peilinz, mirrokni}@google.com

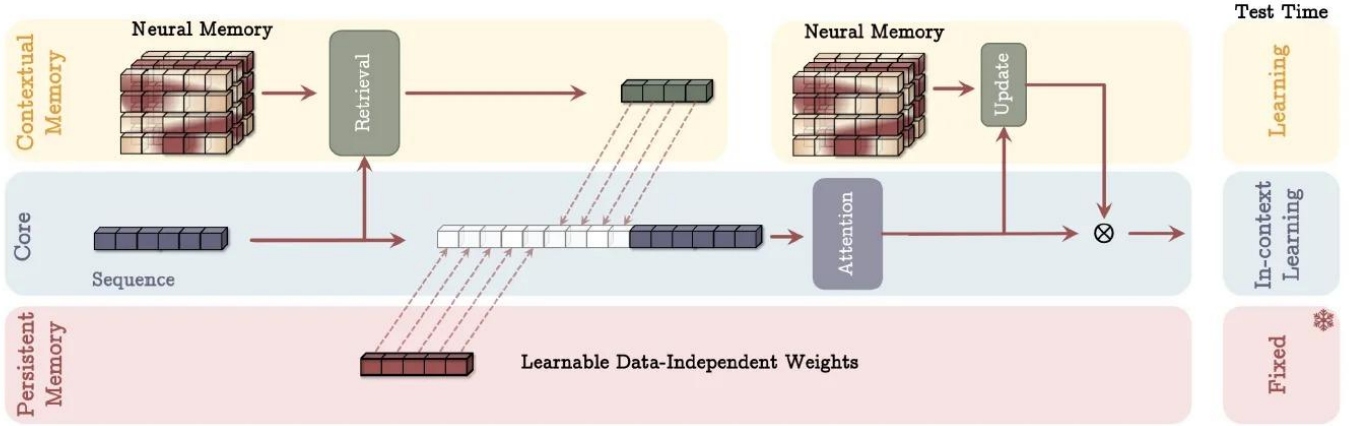


Figure 2: **Memory as a Context (MAC) Architecture.** This architecture includes three branches of (1) core, (2) contextual (long-term) memory, and (3) persistent memory. The core branch concatenates the *corresponding* long-term and persistent memories with the input sequence. Next, attention performs on the sequence and decides what part of the information should store in the long-term memory. At the test time, parameters corresponds to contextual memory are still learning, parameters corresponds to the core branch are responsible for in-context learning, and parameters of persistent memory are responsible to store the knowledge about tasks and so are fixed.

https://arxiv.org/abs/2501.00663

Hugging Face – Um Ecossistema Aberto de Transformers.

Hugging Face é uma plataforma e comunidade de IA que simplifica a adoção de modelos pré-treinados.

- **Bibliotecas**: *transformers*, *datasets* e *tokenizers*.

```
pip install transformers
pip install datasets
...
```

- **Hub**: Repositório colaborativo com modelos e datasets prontos a usar.

https://huggingface.co



Lab

- Lab 03.1 – Transformers
  - ex_01_modelo_BERT_BASE.py

Arquitecturas de Transformers

ENCODER-DECODER

CAPACIDADES PRINCIPAIS
- Processamento de Sequência a Sequência
- Compreensão de Contexto e Geração de Saida
- Atenção Bidirecional e Autorregressiva

CASOS DE USO
- Tradução Automática
- Resumo de Texto
- Resposta a Perguntas
- Legenda de Imagem
- Reconhecimento de Fala

EXEMPLOS
- TS

EXEMPLOS
- BART

DECODER-ONLY

CAPACIDADES PRINCIPAIS
- Geração de Texto Autorregressiva
- Previsão do Próximo Token
- Atenção Unidirecional (Causal Masking)

CASOS DE USO
- Geração de Texto
- Chatbots e Assistentes Virtuais
- Geração de Código
- Narrativa e Escrita Criativa

EXEMPLOS
- Série GPT

EXEMPLOS
- LLaMA
- Falcon

ENCODER-ONLY

CAPACIDADES PRINCIPAIS
- Compreensão e Incorporação de Entrada
- Atenção Bidirecional
- Representação Contextual
- Modelagem de Linguagem Mascarada

CASOS DE USO
- Análise de Sentimento
- Reconhecimento de Entidades Nomeadas
- Classificação de Texto
- Recuperação de Informações
- Resposta a Perguntas (SQuAD)

EXEMPLOS
- BERT

EXEMPLOS
- RoBERTa
- DistilBERT

MODELOS DE EMBEDDING

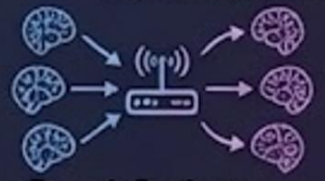


CAPACIDADES PRINCIPAIS
- Converting Text/Data into Vector Representations
- Capturing Semantic Meaning & Similarity

CASOS DE USO
- Semantic Search
- Clustering & Similarity Search
- Recommendation Systems
- Data Processing & Analysis
- Anomaly Detection

EXEMPLOS
- Sentence-BERT
- OpenAI Embedding Models

MIXTURE OF EXPERTS (MoE)

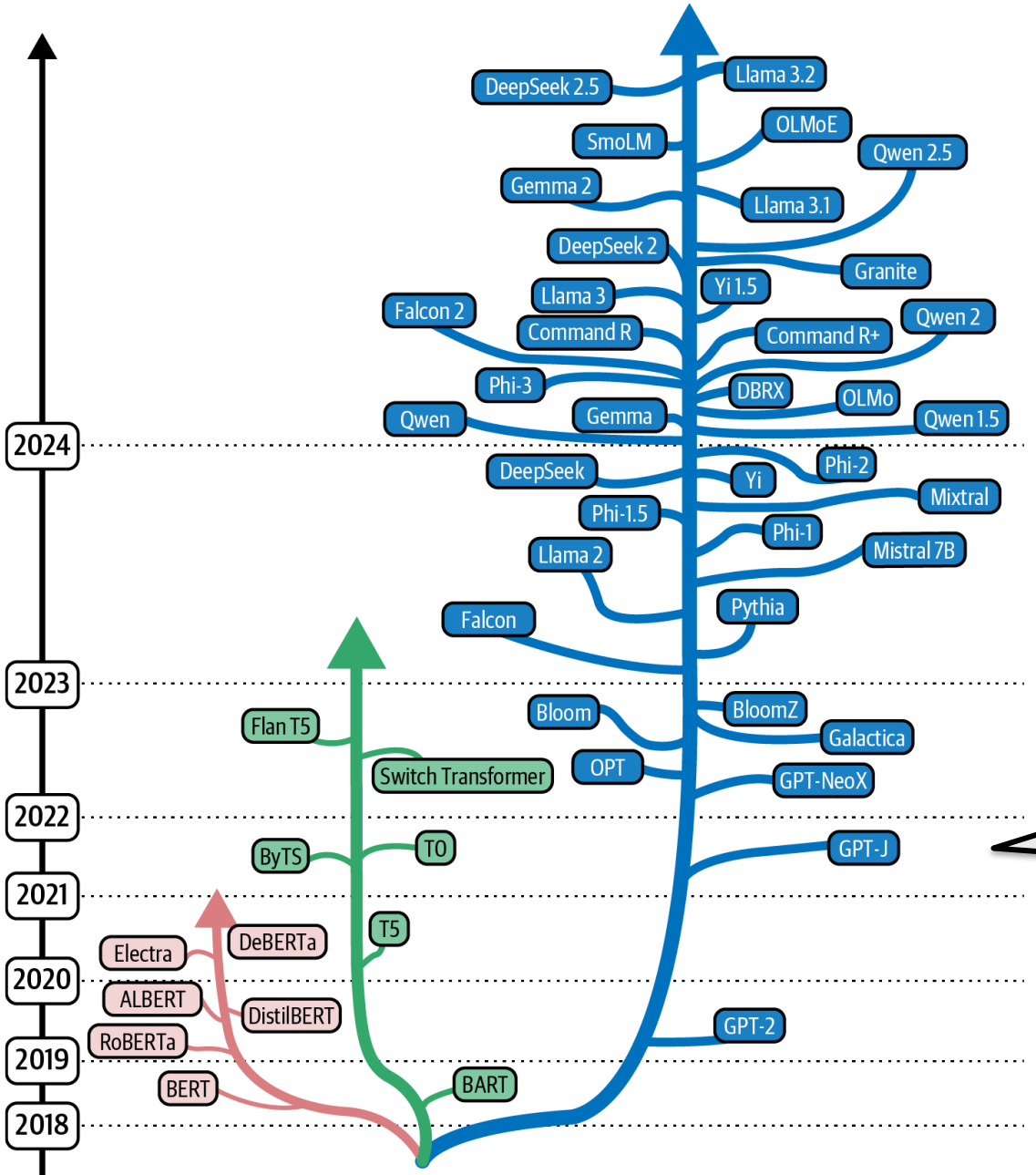


- Dynamic Routing to Specialized Sub-networks (Experts)
- Increased Model Capacity with Less Computational Cost (Sparsity)
- Handling Diverse Tasks

CASOS DE USO
- Large Language Models with Enhanced Performance
- Multimodal Tasks
- Scalable & Efficient Training
- Complex Reasoning

EXAMPLOS
- Mistral 8x7B
- Switch Transformer
- GLaM

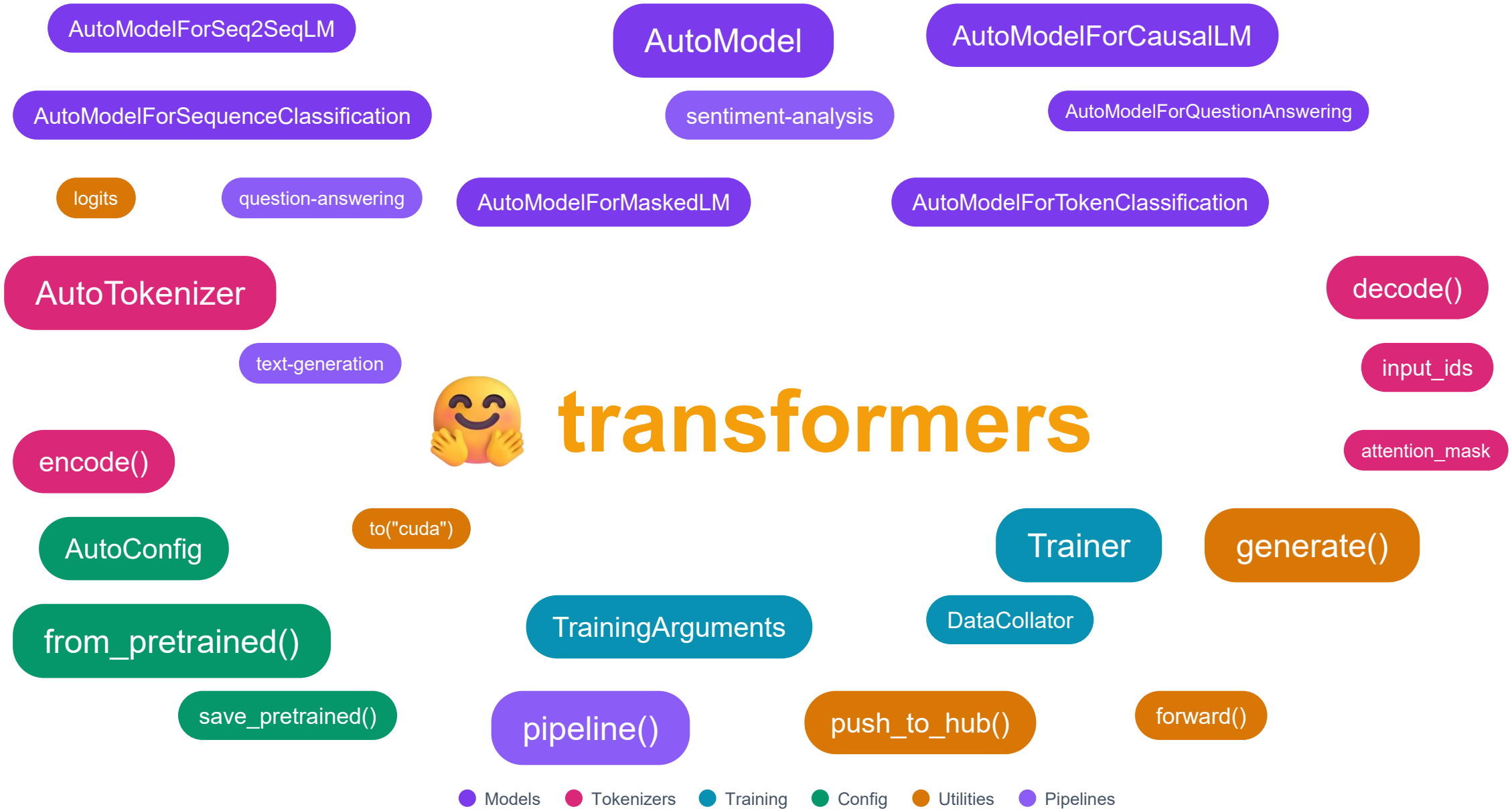
Arquitectura de Transformers dominante



- A arquitetura **padrão** de Transformer é um modelo **decoder only**
- **Alguns modelos usam** apenas o **encoder** ou o **encoder-decoder**

a Vermelho – encoder-only
a Verde – encoder-decoder
a *Azul* – decoder only

Hugging Face – Biblioteca transformers



Hugging Face – Biblioteca transformers - Modelos

Categoria	Componente	Descrição
Modelos (Geral)	AutoModel	Seleciona automaticamente a arquitetura correta baseada no nome do checkpoint. Retorna um modelo base sem nenhuma "cabeça" (head) de tarefa específica.
Modelos (Tarefas)	AutoModelForSequenceClassification	Modelo com uma cabeça de classificação para tarefas como análise de sentimentos ou categorização de texto.
	AutoModelForTokenClassification	Modelo para tarefas ao nível do token, como Reconhecimento de Entidades (NER) ou etiquetagem gramatical (POS tagging).
	AutoModelForQuestionAnswering	Modelo desenhado para QA extrativo — encontra o trecho da resposta dentro de um contexto fornecido.
	AutoModelForCausalLM	Modelo para geração de texto (estilo GPT); prevê o próximo token da esquerda para a direita.
	AutoModelForSeq2SeqLM	Modelo codificador-descodificador (encoder-decoder) para tarefas como tradução ou resumo (ex: T5, BART).
	AutoModelForMaskedLM	Modelo para modelagem de linguagem mascarada (estilo BERT); prevê tokens ocultos (masked).

Hugging Face – Biblioteca transformers – Tokenizadores, Treino e Configuração

Categoria	Componente	Descrição
Tokenizadores	AutoTokenizer	Carrega automaticamente o tokenizador correto para um determinado checkpoint de modelo.
	tokenizer.encode()	Converte texto cru em IDs numéricos de tokens.
	tokenizer.decode()	Converte IDs de tokens de volta para texto legível.
	tokenizer()	Função chamável que retorna um dicionário com input_ids, attention_mask, ...
Treino	Trainer	API de treino de alto nível que gere o ciclo de treino, avaliação, registos (logging) e checkpoints.
	TrainingArguments	Classe de configuração para o Trainer (define tamanho do lote, taxa de aprendizagem, épocas, diretório de saída, etc.).
	DataCollator	Prepara lotes (batches) para treino, aplicando padding/truncagem nas sequências dinamicamente.
Configuração	AutoConfig	Carrega a configuração do modelo (tamanho da camada oculta, número de camadas, tamanho do vocabulário, etc.).
	from_pretrained()	Método de classe para carregar pesos/tokenizadores/configs pré-treinados do Hugging Face Hub ou de um caminho local.
	save_pretrained()	Guarda o modelo, tokenizador ou configuração num diretório local.

Hugging Face – Biblioteca transformers – Pipelines e Utilitários

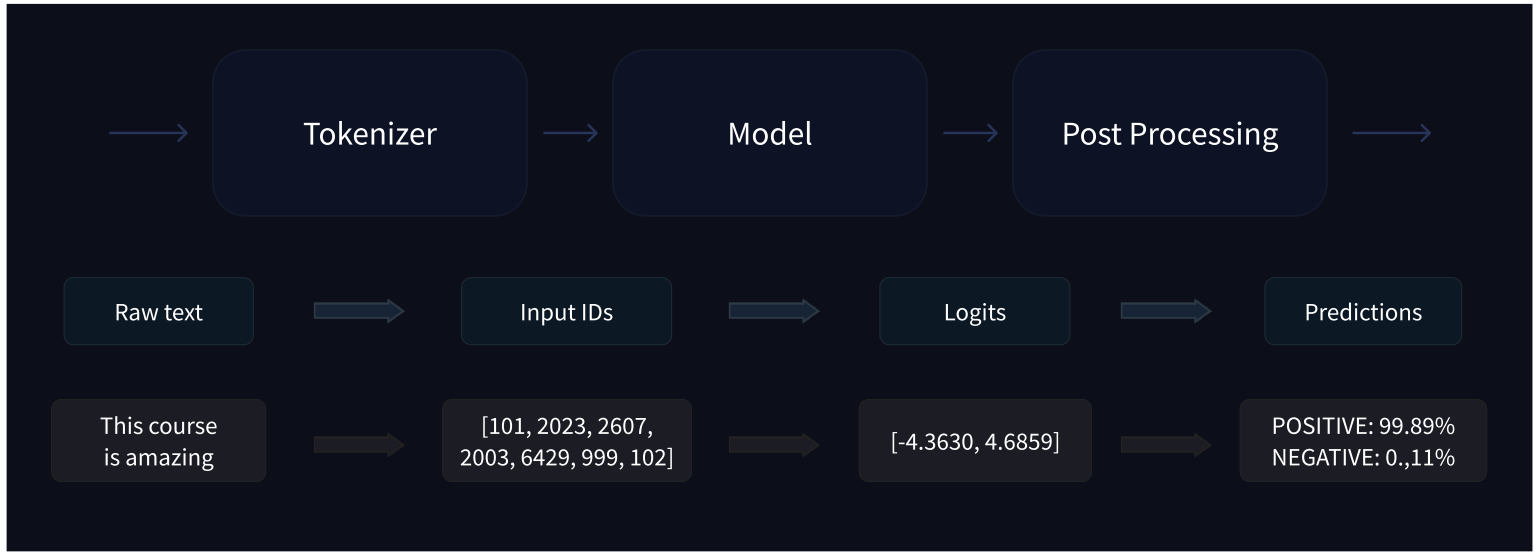
Categoria	Componente	Descrição
Pipelines	pipeline()	API de alto nível para inferência. Abstrai a tokenização, o passo "forward" do modelo e o pós-processamento numa única chamada. Ex: pipeline("sentiment-analysis").
Utilitários	model.generate()	Gera texto para modelos causais/seq2seq com opções como beam search, amostragem, top-k, top-p, etc.
	model.forward()	O passo principal de propagação — recebe inputs tokenizados e retorna as saídas do modelo (logits, estados ocultos).
	push_to_hub()	Faz o upload do seu modelo/tokenizador para o Hugging Face Hub para partilha.
	model.to("cuda")	Move o modelo para a GPU para inferência ou treino mais rápidos.

Hugging Face Pipelines - para inferência

- Um [pipeline] simplifica o uso dos modelos no Model Hub (https://huggingface.co/models).

- Mesmo que não se defina, o pipeline() **carrega automaticamenta** um **modelo predeterminado** e um **tokenizador** com capacidade de inferência para a tarefa em causa:
  - geração de texto
  - segmentação de imagens
  - classificação de áudio
  - etc

(dentro da pipeline)



Hugging Face Pipelines - task

O **pipeline()** faz muitas tarefas em diferentes modalidades, **algumas** das quais:

```
>> from transformers import pipeline
```

Chamada **simples** para um único item:

```
>> pipe = pipeline("text-classification")
>> pipe("This restaurant is awesome")
```

```
[{'label': 'POSITIVE', 'score': 0.9998743534088135}]
```

Se pretender **utilizar um modelo específico** do hub, pode ignorar a tarefa no caso de o modelo já definir:

```
>> pipe = pipeline(model="FacebookAI/roberta-large-mnli")
>> pipe("This restaurant is awesome")
```

```
[{'label': 'NEUTRAL', 'score': 0.7311364583969912}]
```

Para chamar **um pipeline em vários itens**, pode chamá-lo com uma lista.

```
>> pipe = pipeline("text-classification")
>> pipe(["This restaurant is awesome",
      "This restaurant is awful"])
```

```
[{'label': 'POSITIVE', 'score': 0.9998743534088135},
 {'label': 'NEGATIVE', 'score': 0.9996669292449951}]
```

Task	Description	Modality	Pipeline identifier
Text classification	assign a label to a given sequence of text	NLP	pipeline(task="sentiment-analysis")
Text generation	generate text given a prompt	NLP	pipeline(task="text-generation")
Summarization	generate a summary of a sequence of text or document	NLP	pipeline(task="summarization")
Image classification	assign a label to an image	Computer vision	pipeline(task="image-classification")
Image segmentation	assign a label to each individual pixel of an image (supports semantic, panoptic, and instance segmentation)	Computer vision	pipeline(task="image-segmentation")
Object detection	predict the bounding boxes and classes of objects in an image	Computer vision	pipeline(task="object-detection")
Audio classification	assign a label to some audio data	Audio	pipeline(task="audio-classification")
Automatic speech recognition	transcribe speech into text	Audio	pipeline(task="automatic-speech-recognition")
Visual question answering	answer a question about the image, given an image and a question	Multimodal	pipeline(task="vqa")
Document question answering	answer a question about the document, given a document and a question	Multimodal	pipeline(task="document-question-answering")
Image captioning	generate a caption for a given image	Multimodal	pipeline(task="image-to-text")

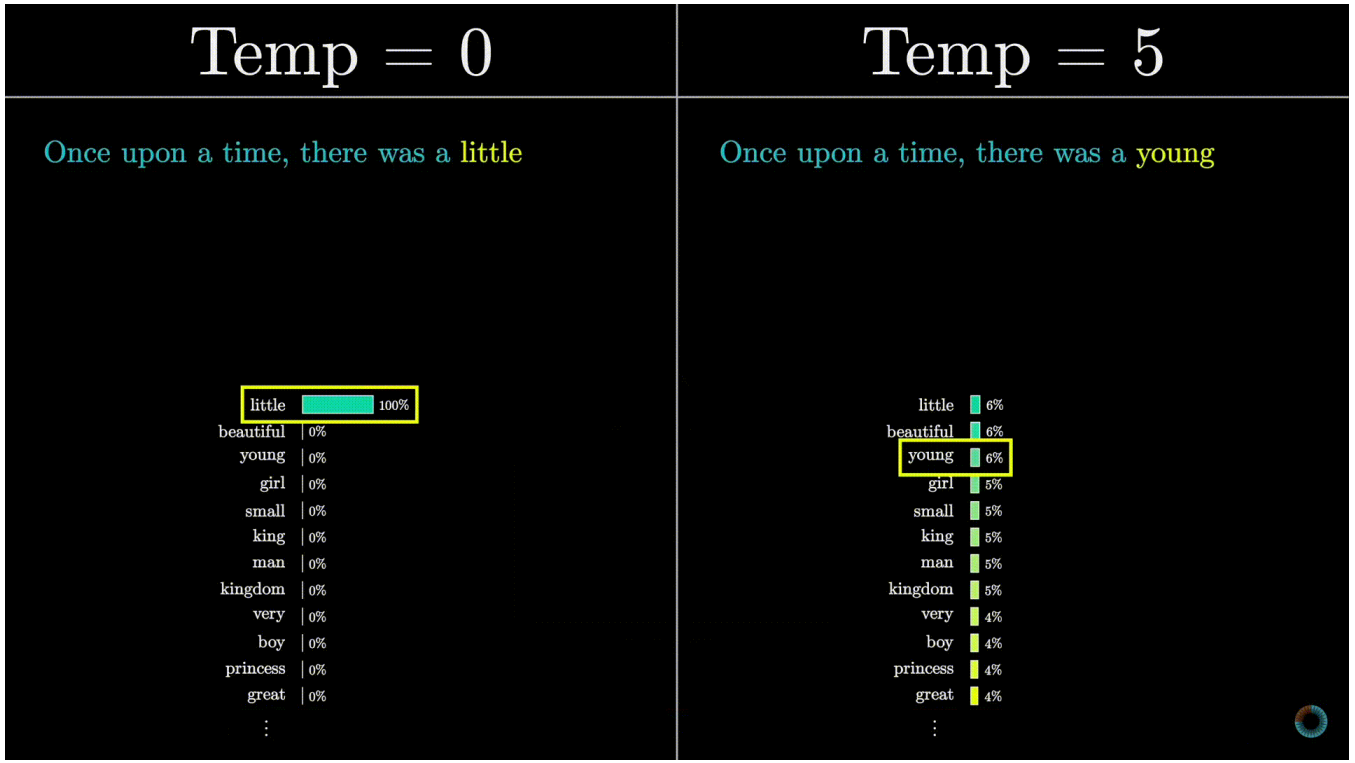
Hugging Face Pipelines – parâmetros de Coerência e Criatividade

Parâmetro	Ex.	Explicação Simples
temperature	0.7	Controla o grau de criatividade. Se for mais alto (ex.: 1.0, 1.2), o modelo arrisca palavras menos prováveis (fica mais “criativo”). Se for mais baixo (ex.: 0.2), o modelo torna-se mais previsível e repete-se mais.

Temperatura num Transformer?

Parâmetro que controla a **aleatoriedade na geração de texto**.

- a temperatura ideal depende do equilíbrio desejado entre criatividade e coerência



Valores **mais altos (>1)** tornam as respostas mais criativas e imprevisíveis.
Valores **mais baixos (<1)** tornam-nas mais conservadoras e repetitivas.

Hugging Face Pipelines – parâmetros de Coerência e Criatividade

Parâmetro	Ex.	Explicação Simples
temperature	0.7	Controla o grau de criatividade. Se for mais alto (ex.: 1.0, 1.2), o modelo arrisca palavras menos prováveis (fica mais “criativo”). Se for mais baixo (ex.: 0.2), o modelo torna-se mais previsível e repete-se mais.
top_k	50	O modelo escolhe entre as 50 palavras mais prováveis em cada passo, em vez de todas as possíveis. Isto reduz a probabilidade de gerar algo muito estranho. Quanto menor o número, mais “limitada” fica a escolha.
top_p	0.9	Chama-se “nucleus sampling”: o modelo só pode escolher palavras que, juntas, cobrem 90% da probabilidade total. Ajuda a evitar palavras raríssimas e sem sentido.
repetition_penalty	1.2	Se o valor for maior que 1, o modelo recebe uma “penalização” sempre que repete palavras ou expressões. Ajuda a evitar repetições incansáveis (loops).
no_repeat_ngram_size	2	Impede o modelo de repetir a mesma sequência de 2 (ou 3, 4, etc.) palavras várias vezes seguidas. Útil para não criar frases como “Ok, ok, ok, ok...”.
do_sample	True	Se True, o modelo usa aleatoriedade para escolher as palavras. Se for False, escolhe sempre a palavra mais provável (ficando mais “robótico” e repetitivo).

Hugging Face Pipelines – parâmetros de Tamanho e Estratégia de Geração

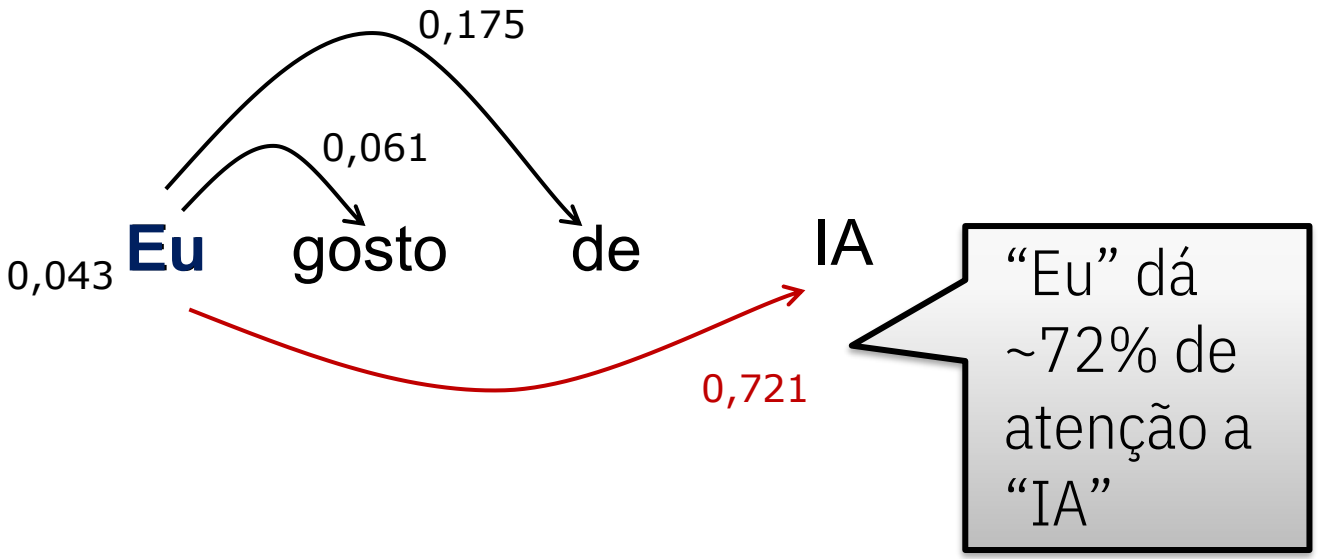
Parâmetro	Ex.	Explicação Simples
max_new_tokens	50	Número de novas palavras (tokens) que o modelo pode escrever para além do teu texto de entrada (prompt). Exemplo: se for 50, ele não passará disso ao responder.
max_length	128	Comprimento total (incluindo o prompt) que o modelo poderá ter na resposta final. Se o prompt for já muito grande, este valor pode ser insuficiente e o modelo não terá espaço para gerar o texto pretendido.
num_beams	1	Quando > 1, o modelo usa um método chamado “beam search”, que analisa várias hipóteses em paralelo para tentar escolher a frase “mais correta”. Mas com 1 beam, ele gera texto de forma mais direta (busca gulosa).
early_stopping	False	Em “beam search”, se for True, pára de gerar assim que todas as hipóteses cheguem a um “fim de frase” (token especial). Se for False, força o modelo a usar todo o espaço permitido (até max_length ou max_new_tokens).
pad_token_id	50256	Token de preenchimento (padding). Para modelos GPT-2, geralmente é 50256. É usado para preencher espaço quando o texto é mais curto do que o tamanho esperado.
eos_token_id	50256	Token de fim de frase (“end of sequence”). Diz ao modelo que a geração pode parar. Em GPT-2, é comum usar o mesmo valor de pad_token_id.

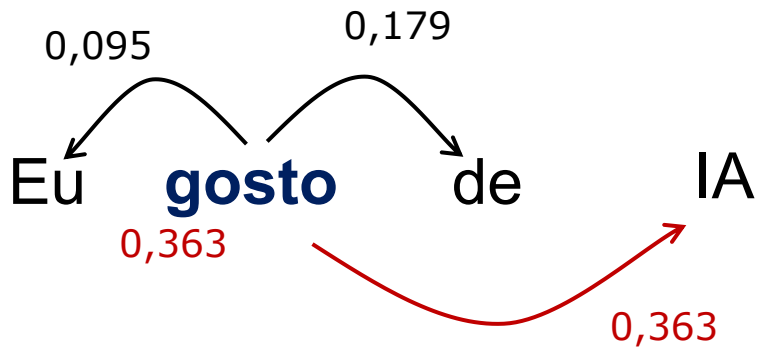
Lab

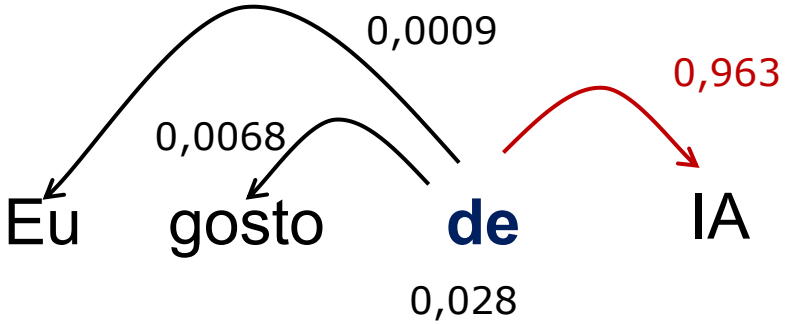
- Lab 03.1 – Transformers
  - ex_02_pipeline1_visao.py
  - ex_03_pipeline2_analise_sentimentos.py
  - ex_04_pipeline3_audio.py
  - ex_05_pipeline4_generate_music.py

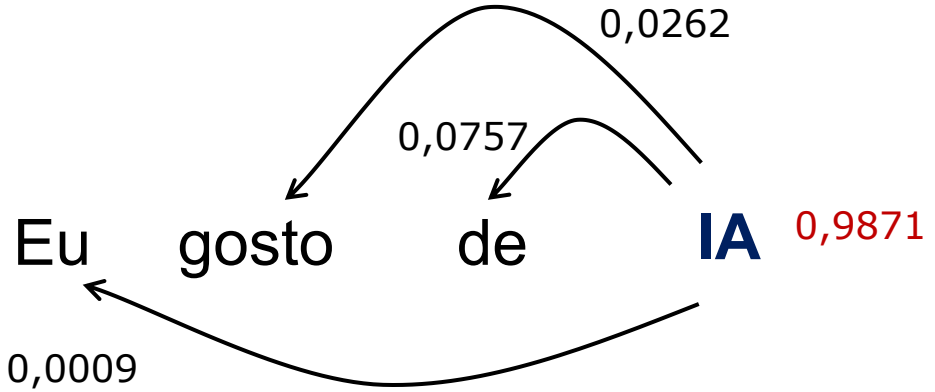
Self-Attention

Self-attention é um mecanismo que permite a um modelo **ponderar a importância de diferentes palavras** numa sequência **em relação umas às outras**.

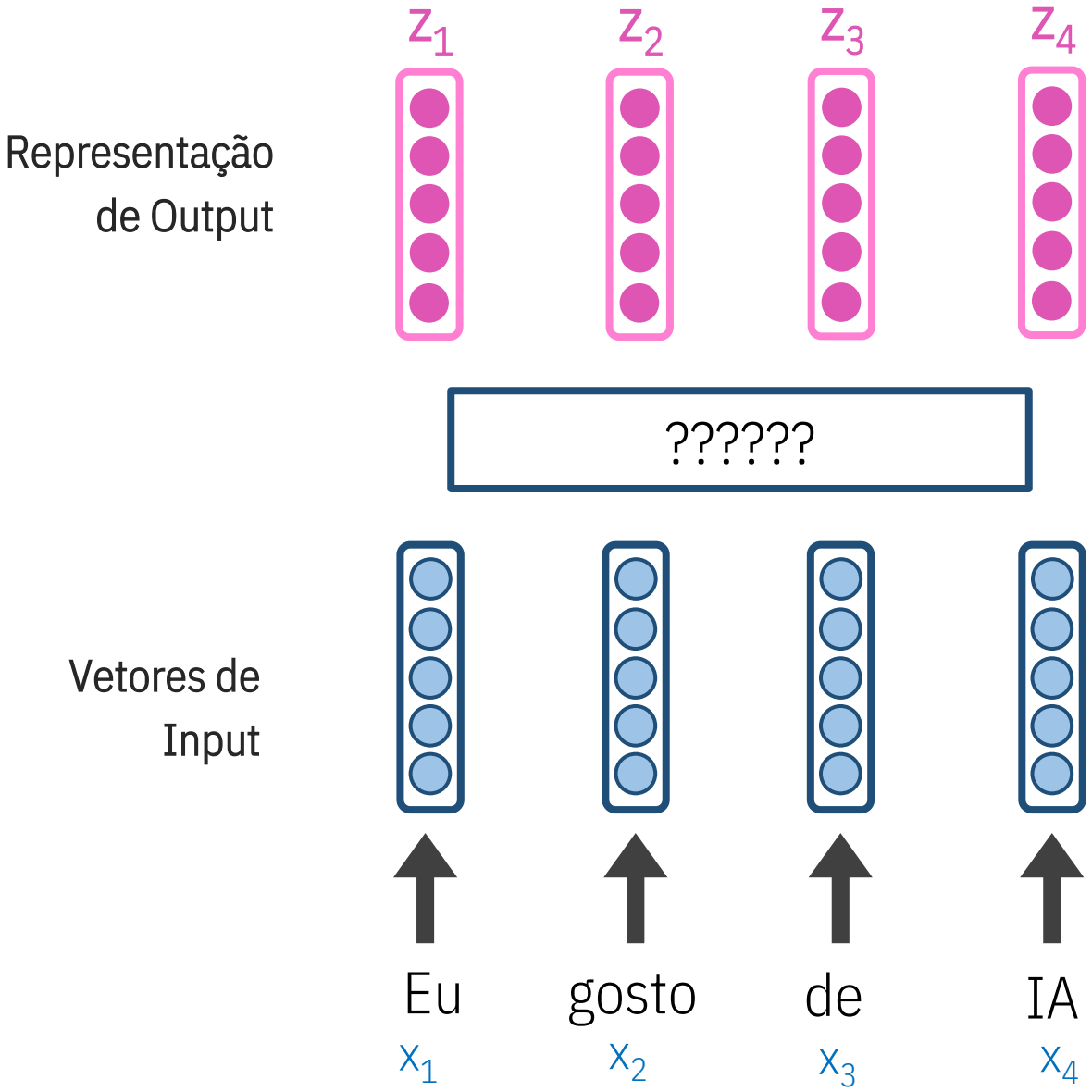






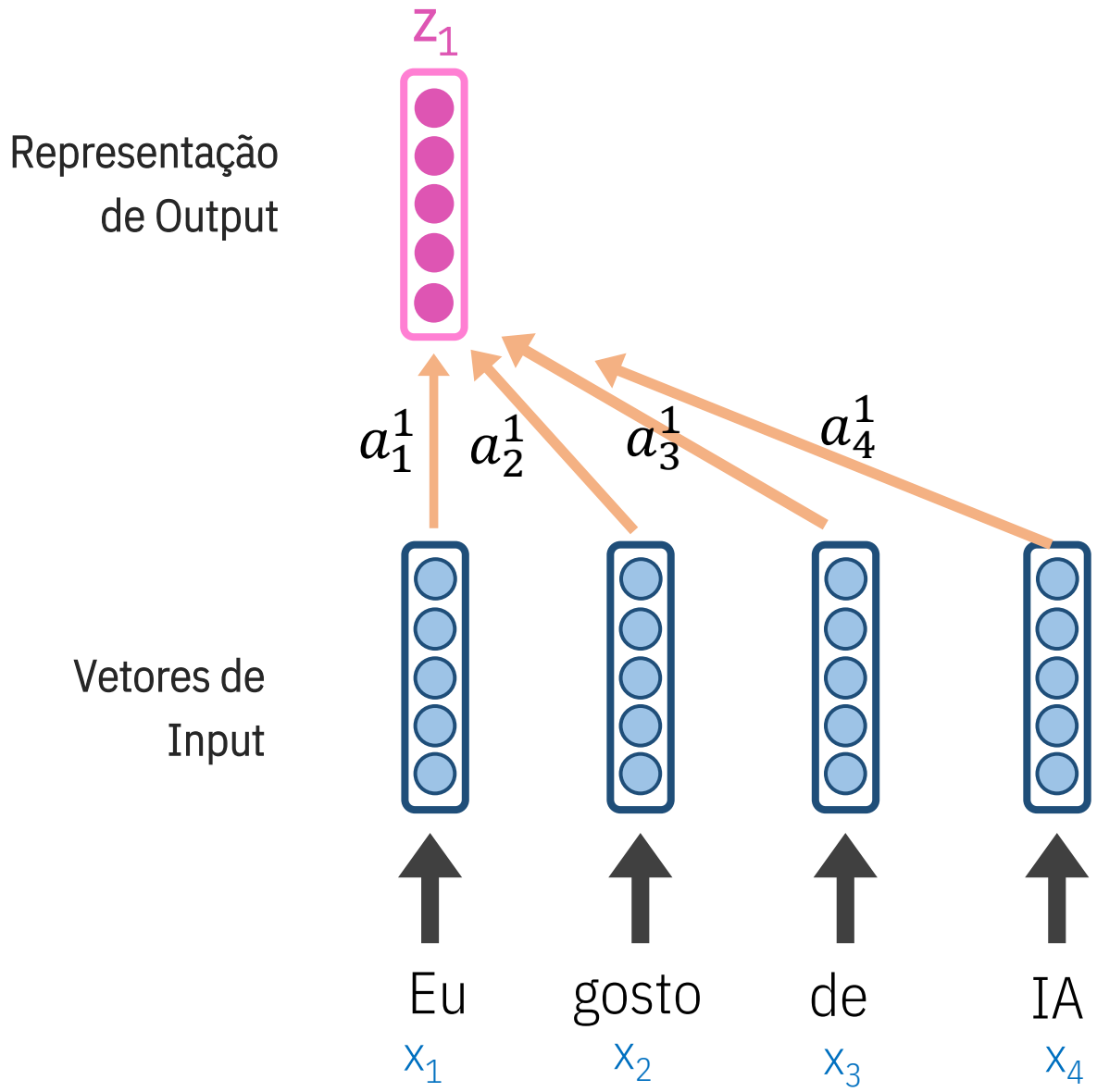


Self-Attention



O objectivo do **Self-Attention** é criar grandes representações, z_i, do input

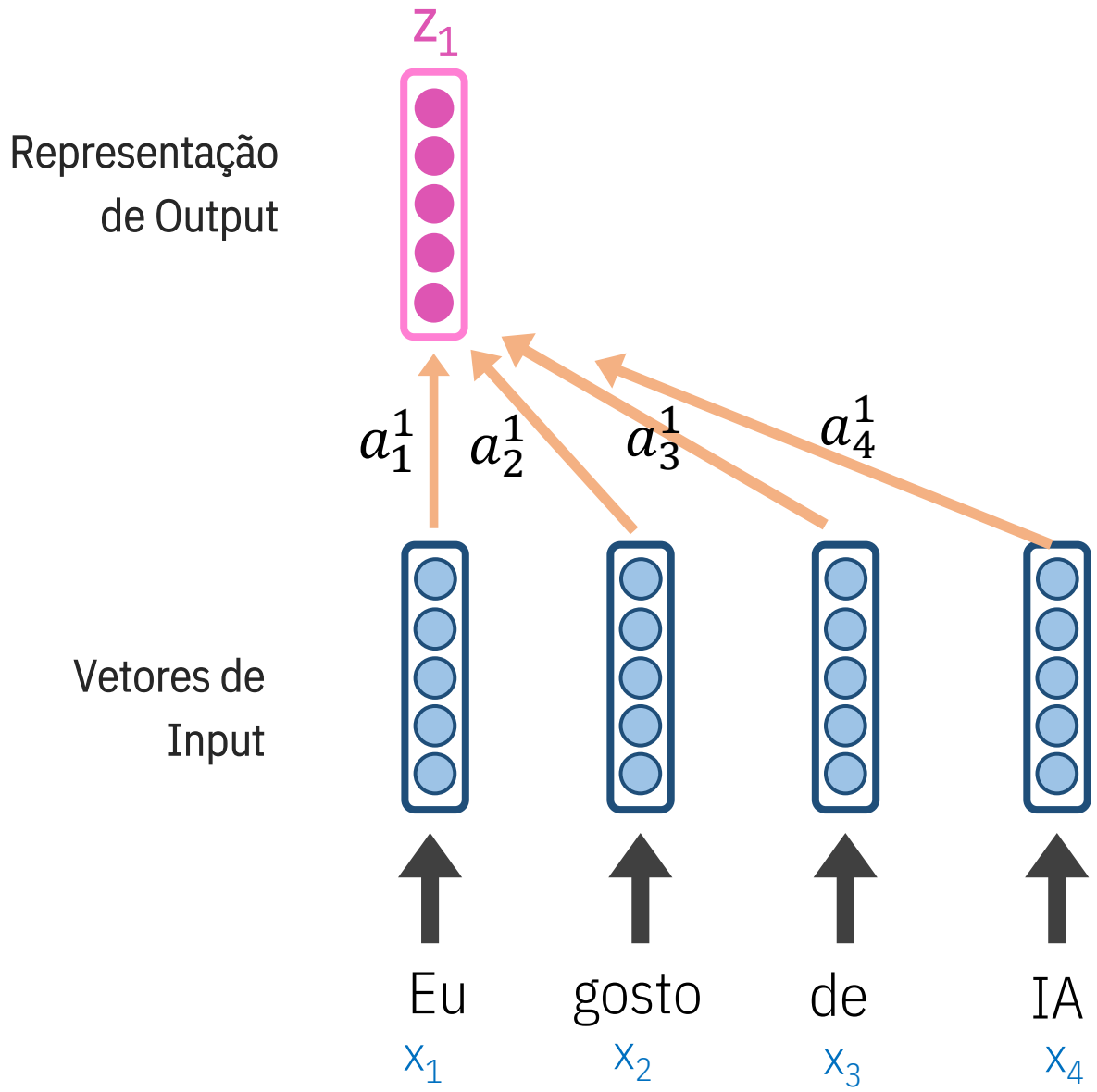
Self-Attention



O objectivo do **Self-Attention** é criar grandes representações, z_i, do input

z_1 basear-se-á numa contribuição pesada de x_1, x_2, x_3, x_4

Self-Attention

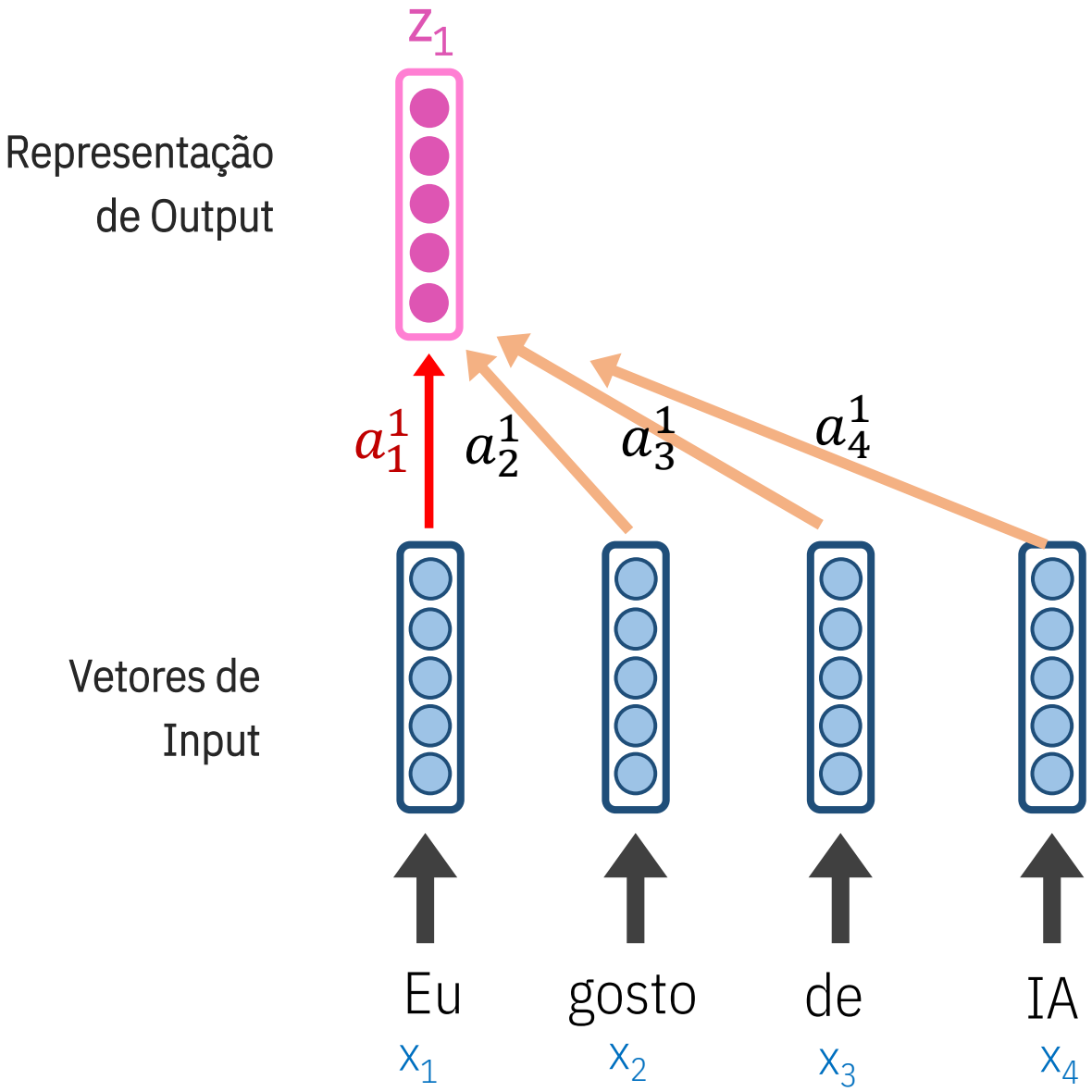


O objectivo do **Self-Attention** é criar grandes representações, z_i, do input

z_1 basear-se-á numa contribuição pesada de x_1, x_2, x_3, x_4

a_i^1 é "só" um peso. Mais está a acompanhar nas entranhas, mas é efetivamente a ponderação de *versões* de x_1, x_2, x_3, x_4

Self-Attention



cada x_i tem 3 pequenos vetores associados. Por exemplo, x_1 tem:

- Query $\mathbf{q}_i$
- Key $\mathbf{k}_i$
- Value $\mathbf{v}_i$

Self-Attention

Passo 1: Uma Self-Attention Head tem apenas 3 matrizes de peso W_q, W_k, W_v no total. As mesmas 3 matrizes de peso são multiplicadas por cada x_i para criar todos os vetores:

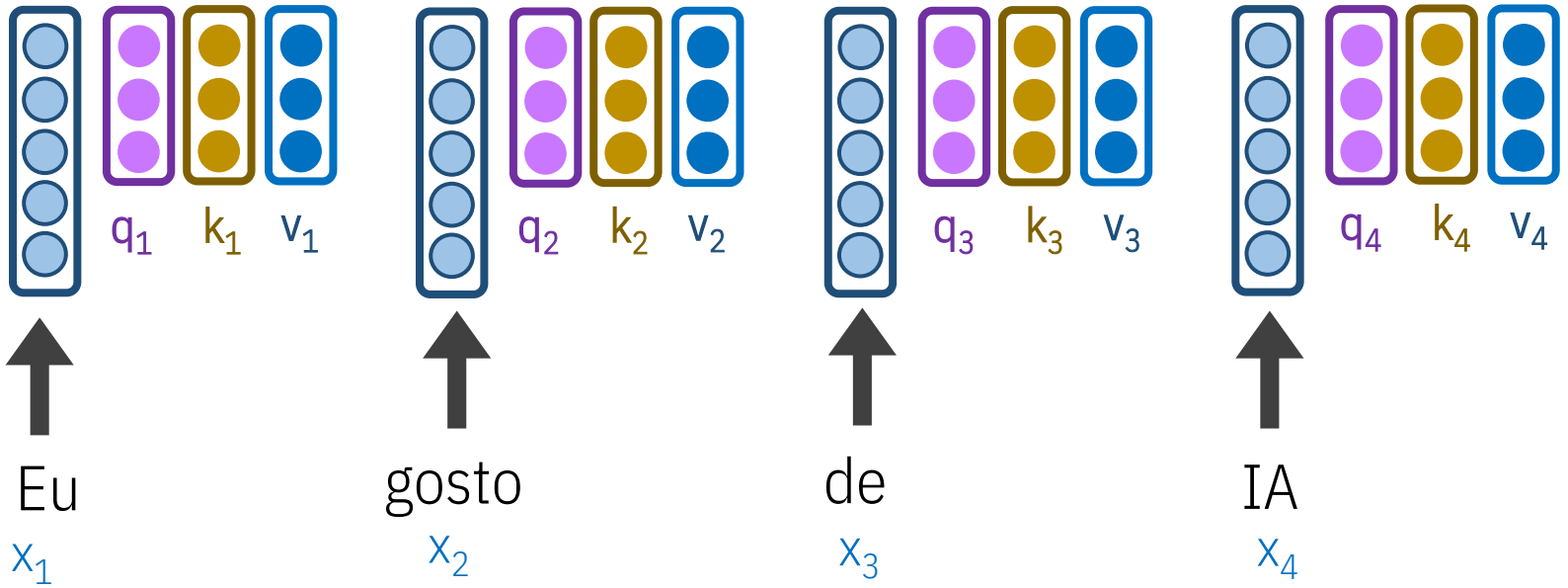
$$q_i = w_q x_i$$

$$k_i = w_k x_i$$

$$v_i = w_v x_i$$

cada x_i tem 3 pequenos vetores associados. Por exemplo, x_1 tem:

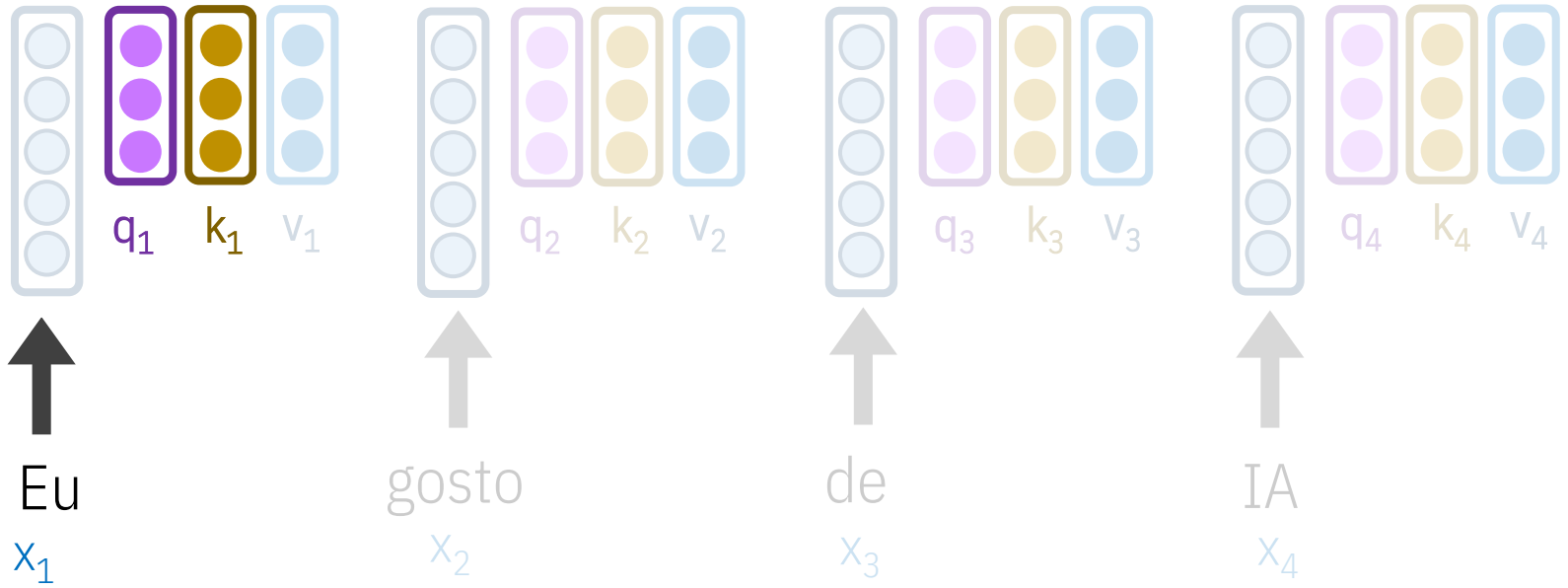
- Query q_1
- Key k_1
- Value v_1



Self-Attention

Passo 2: Para a palavra x_1, vamos calcular os scores s_1, s_2, s_3, s_4, que representam a atenção a prestar a cada uma das respetivos ”word” v_i

$s_1 = q_1 \cdot k_1 = 0$

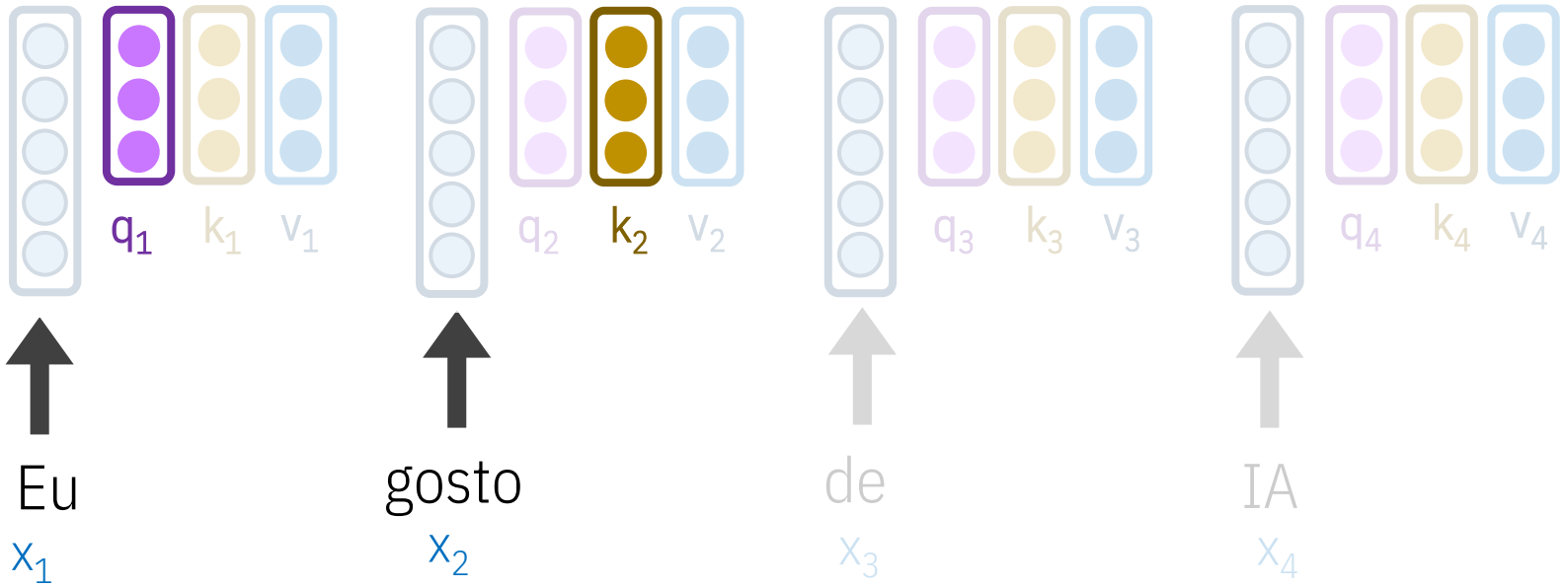


Self-Attention

Passo 2: Para a palavra x_1, vamos calcular os scores s_1, s_2, s_3, s_4, que representam a atenção a prestar a cada uma das respetivos "word" v_i

$s_2 = q_1 \cdot k_2 = 2,8$

$s_1 = q_1 \cdot k_1 = 0$



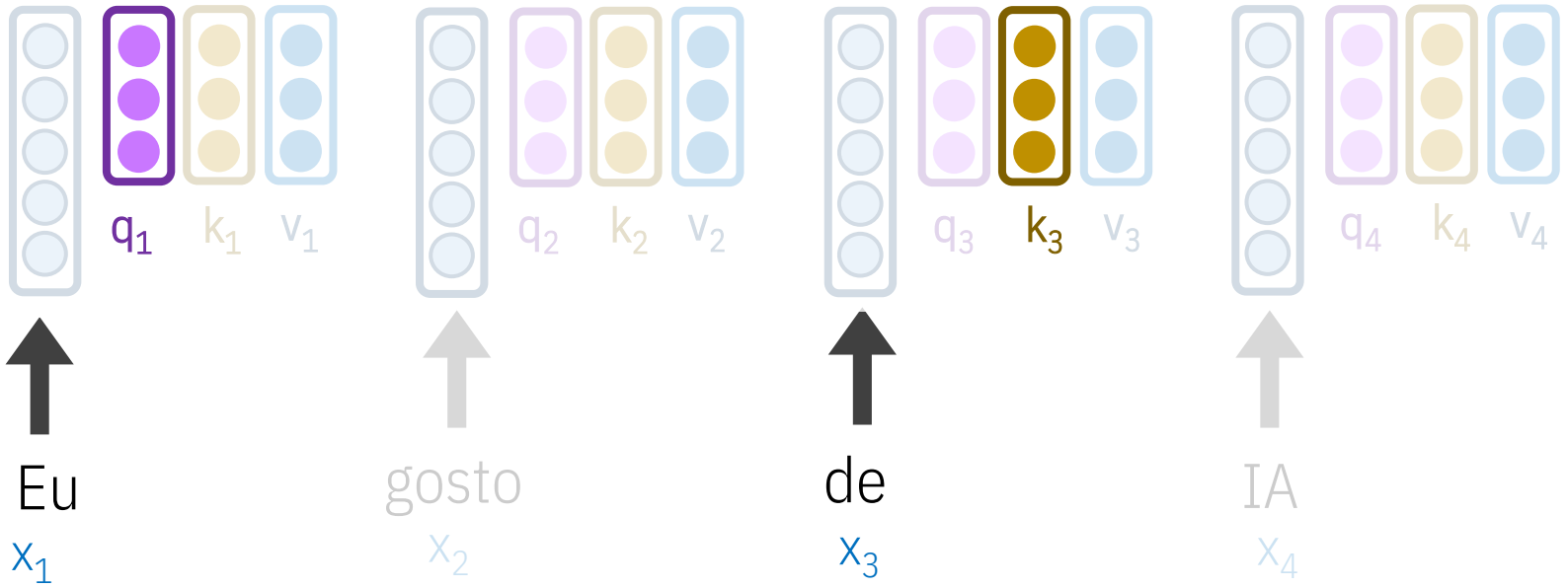
Self-Attention

Passo 2: Para a palavra x_1, vamos calcular os scores s_1, s_2, s_3, s_4, que representam a atenção a prestar a cada uma das respetivos ”word” v_i

$s_3 = q_1 \cdot k_3 = 11,23$

$s_2 = q_1 \cdot k_2 = 2,8$

$s_1 = q_1 \cdot k_1 = 0$



Self-Attention

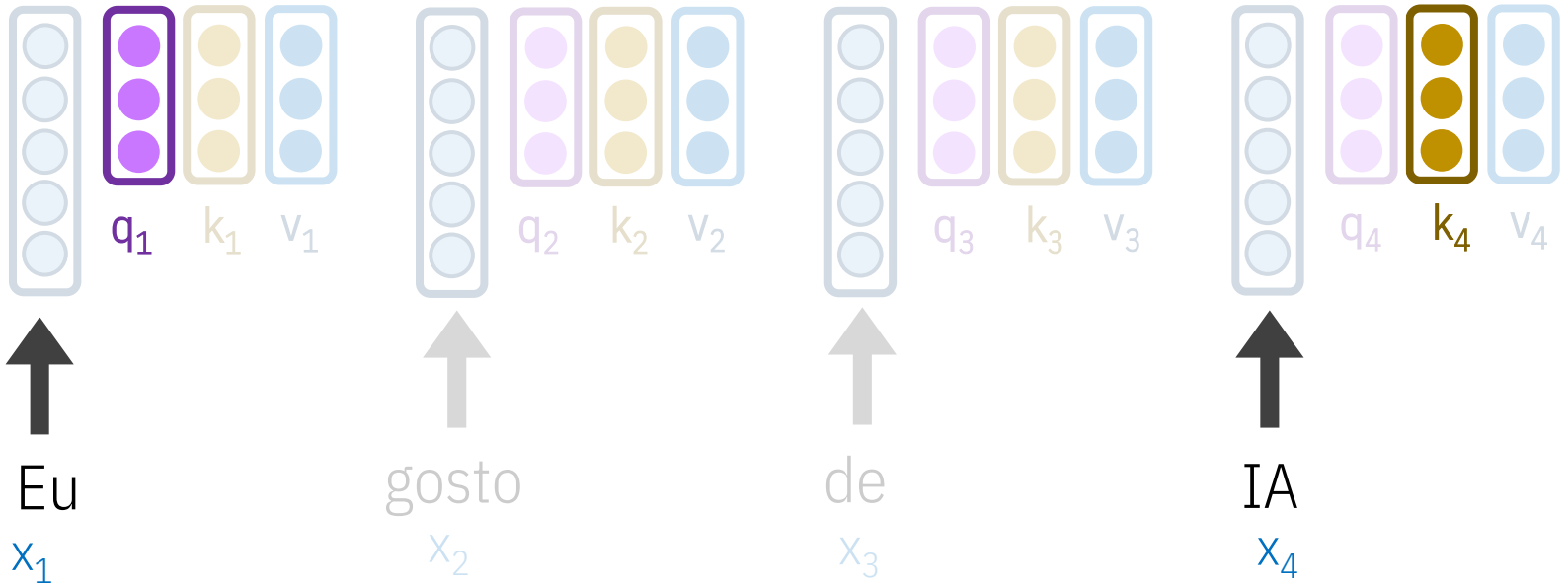
Passo 2: Para a palavra x_1, vamos calcular os scores s_1, s_2, s_3, s_4, que representam a atenção a prestar a cada uma das respetivos ”word” v_i

$s_4 = q_1 \cdot k_4 = 22{,}56$

$s_3 = q_1 \cdot k_3 = 11{,}23$

$s_2 = q_1 \cdot k_2 = 2{,}8$

$s_1 = q_1 \cdot k_1 = 0$



Self-Attention

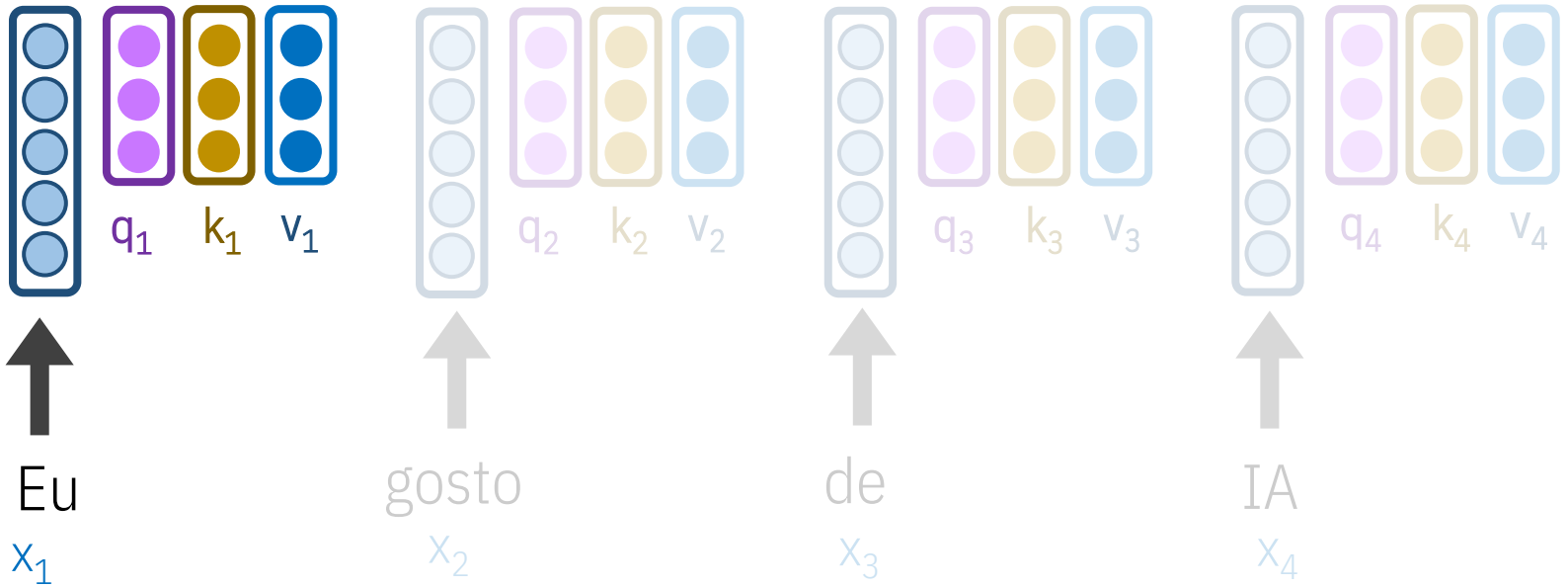
Passo 3: Os nossos scores s_1, s_2, s_3, s_4 não somam 1. Vamos dividir por $\sqrt{len(k_i)}$ e aplicar softmax .

$s_4 = q_1 \cdot k_4 = 22{,}56$ $\quad a_4 = \boldsymbol{\sigma}(s_4/8) = 0{,}721$

$s_3 = q_1 \cdot k_3 = 11{,}23$ $\quad a_3 = \boldsymbol{\sigma}(s_3/8) = 0{,}175$

$s_2 = q_1 \cdot k_2 = 2{,}8$ $\quad a_2 = \boldsymbol{\sigma}(s_2/8) = 0{,}061$

$s_1 = q_1 \cdot k_1 = 0$ $\quad a_1 = \boldsymbol{\sigma}(s_1/8) = 0{,}043$



Self-Attention

Passo 3: Os nossos scores s_1, s_2, s_3, s_4 não somam 1. Vamos dividir por $\sqrt{len(k_i)}$ e aplicar softmax .

$s_4 = q_1 \cdot k_4 = 22{,}56$

$s_3 = q_1 \cdot k_3 = 11{,}23$

$s_2 = q_1 \cdot k_2 = 2{,}8$

$s_1 = q_1 \cdot k_1 = 0$

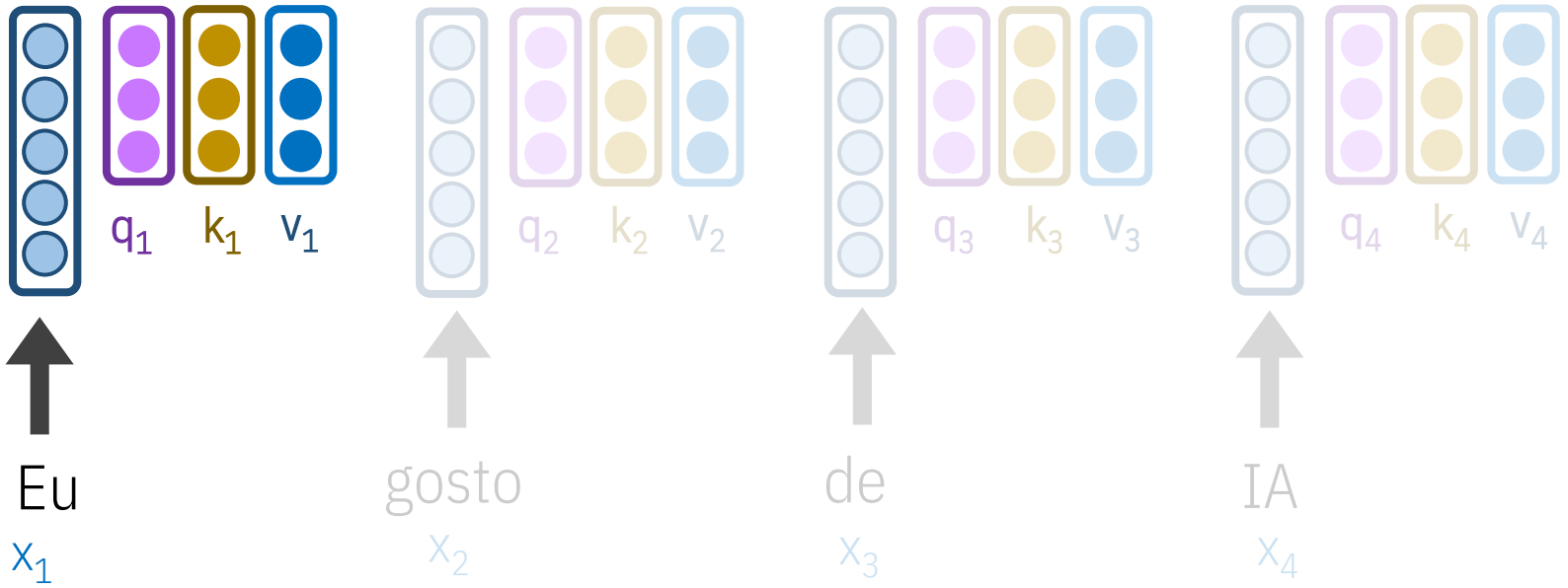
$a_4 = \sigma(s_4/8) = 0{,}721$

$a_3 = \sigma(s_3/8) = 0{,}175$

$a_2 = \sigma(s_2/8) = 0{,}061$

$a_1 = \sigma(s_1/8) = 0{,}043$

Em vez destes valores a_i diretamente pesarem os vetores originais de x_i, eles diretamente pesam os vetores v_i.



Self-Attention

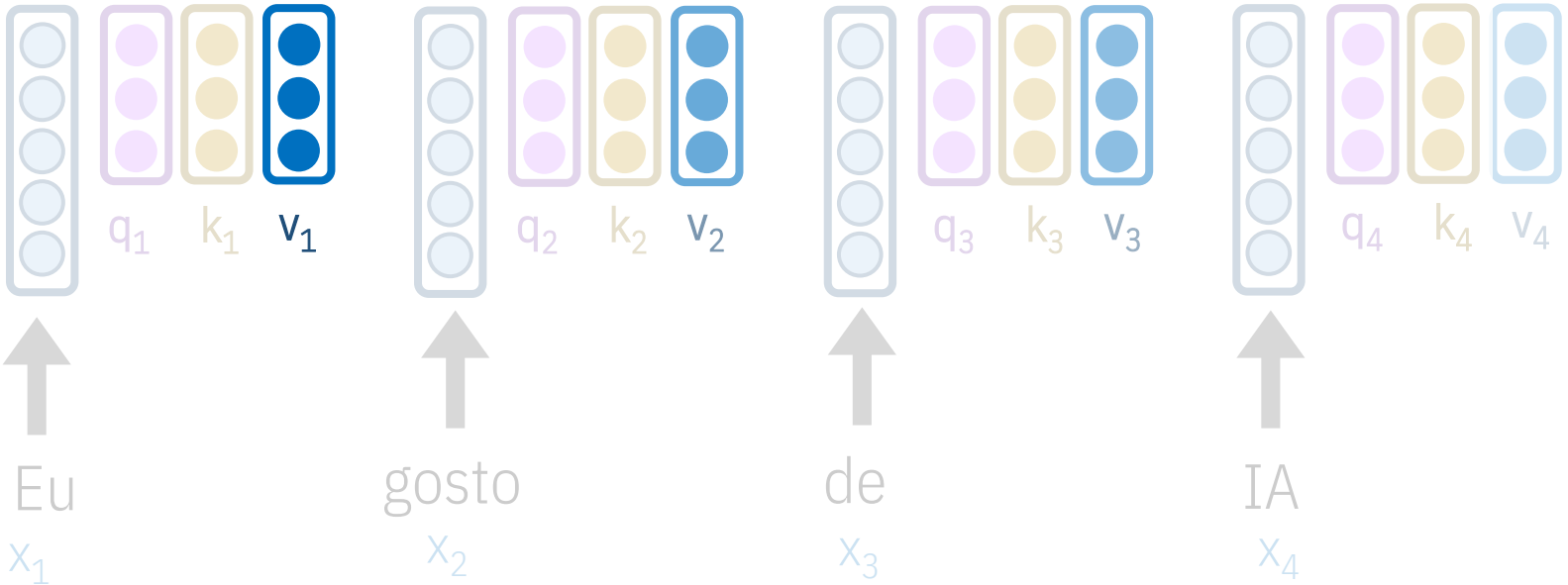
Passo 4: Vamos pesar os nossos vetores v_i e simplesmente resumi-los!



$$z_1 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$

$$= 0{,}043 \cdot v_1 + 0{,}061 \cdot v_2 + 0{,}175 \cdot v_3 + 0{,}721 \cdot v_4$$

"Eu" dá ~72% de atenção a "IA"

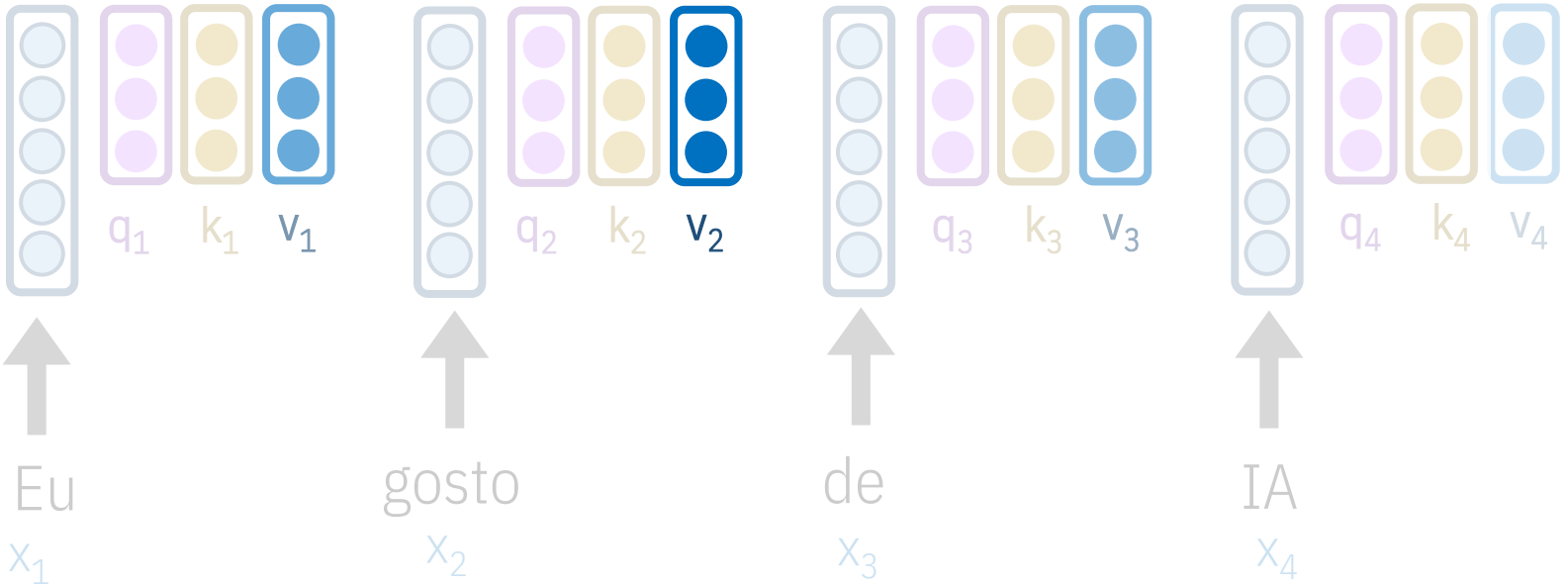


Self-Attention

Passo 5: Repetimos isso para todas as outras palavras, rendendo-nos grandes e novas representações z_i!



$$z_2 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$

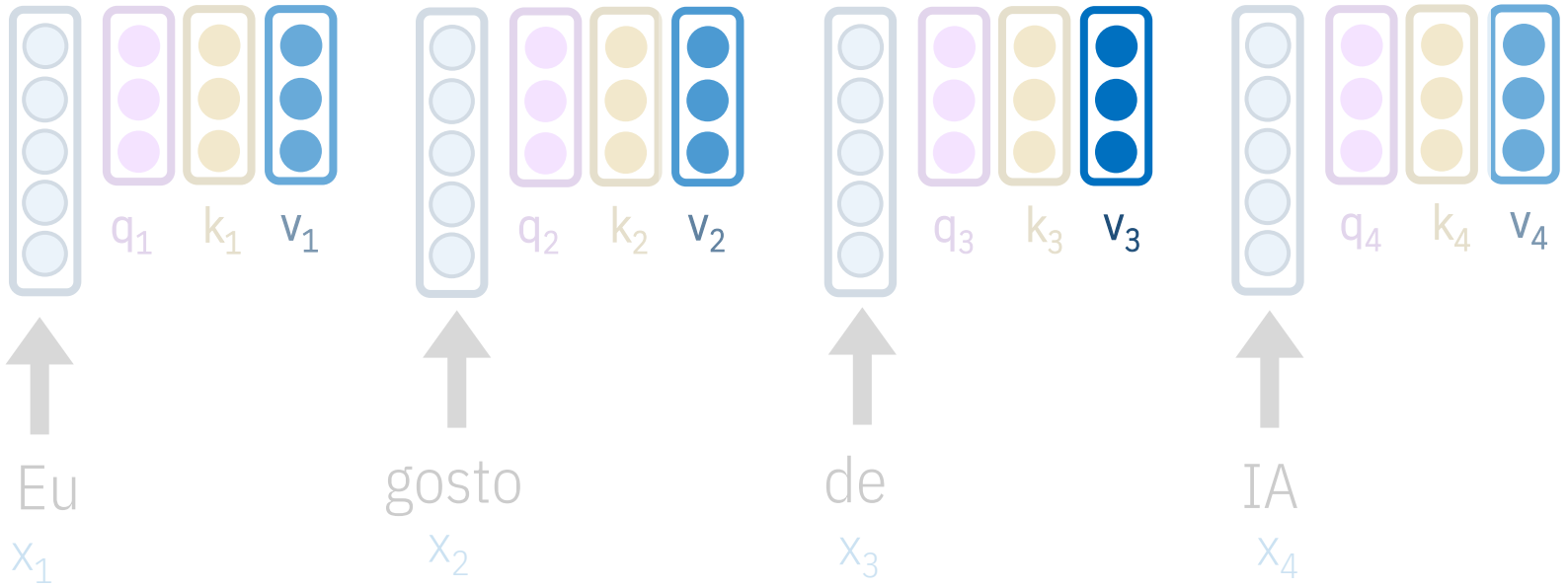


Self-Attention

Passo 5: Repetimos isso para todas as outras palavras, rendendo-nos grandes e novas representações z_i!



$$z_3 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$

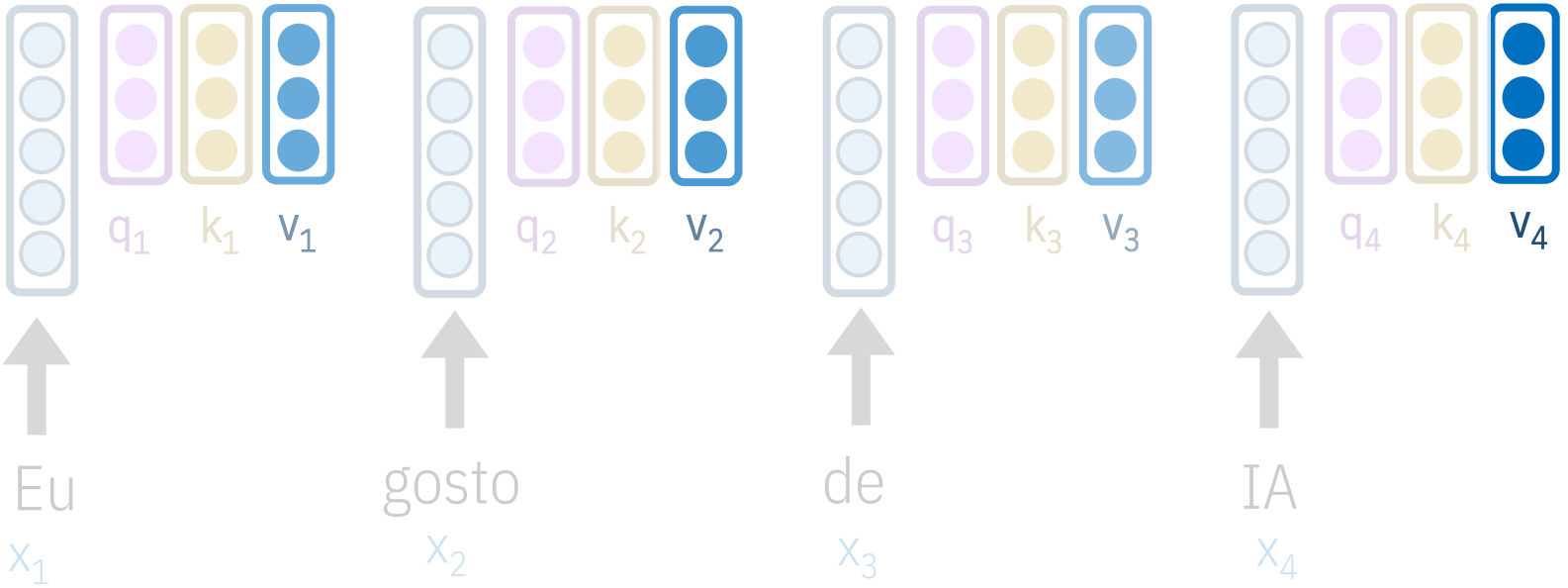


Self-Attention

Passo 5: Repetimos isso para todas as outras palavras, rendendo-nos grandes e novas representações z_i!



$$z_4 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$



Vamos ilustrar outro exemplo:



$$z_2 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$

Lembre-se, usamos as mesmos 3 matrizes de peso

W_q, W_k, W_v como fizemos para a computação z_1.

Isto dá-nos q_2, k_2, v_2

Self-Attention

Passo 1: Uma Self-Attention Head tem apenas 3 matrizes de peso W_q, W_k, W_v no total. As mesmas 3 matrizes de peso são multiplicadas por cada x_i para criar todos os vetores:

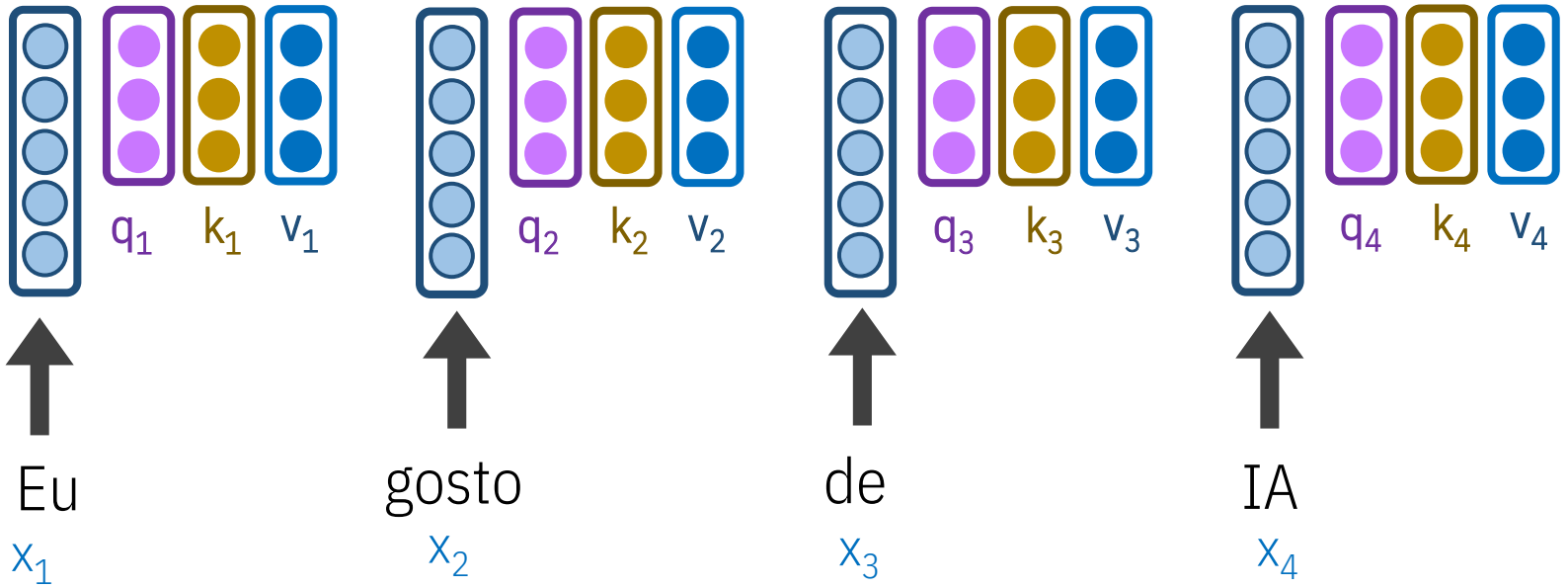
$$q_i = w_q x_i$$

$$k_i = w_k x_i$$

$$v_i = w_v x_i$$

cada x_i tem 3 pequenos vetores associados. Por exemplo, x_1 tem:

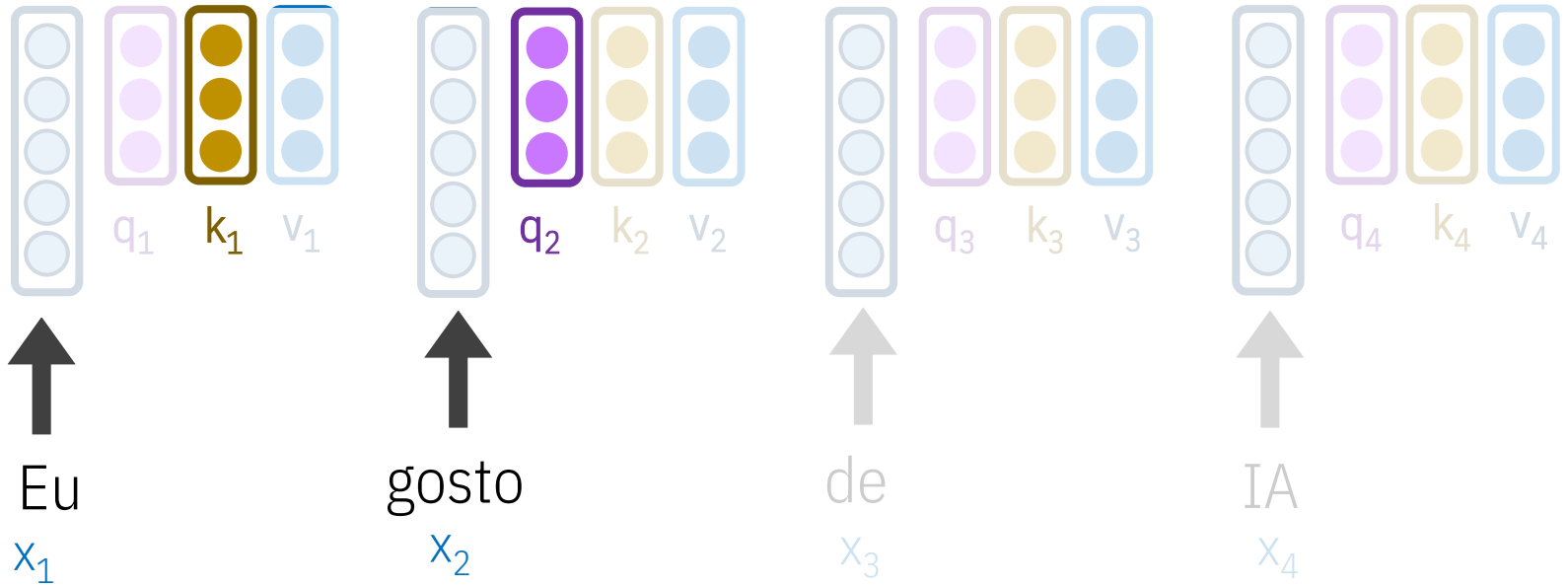
- Query q_1
- Key k_1
- Value v_1



Self-Attention

Passo 2: Para a palavra x_2, vamos calcular os scores s_1, s_2, s_3, s_4, que representam a atenção a prestar a cada uma das respetivos ”word” v_i

$s_1 = q_2 \cdot k_1 = 0$

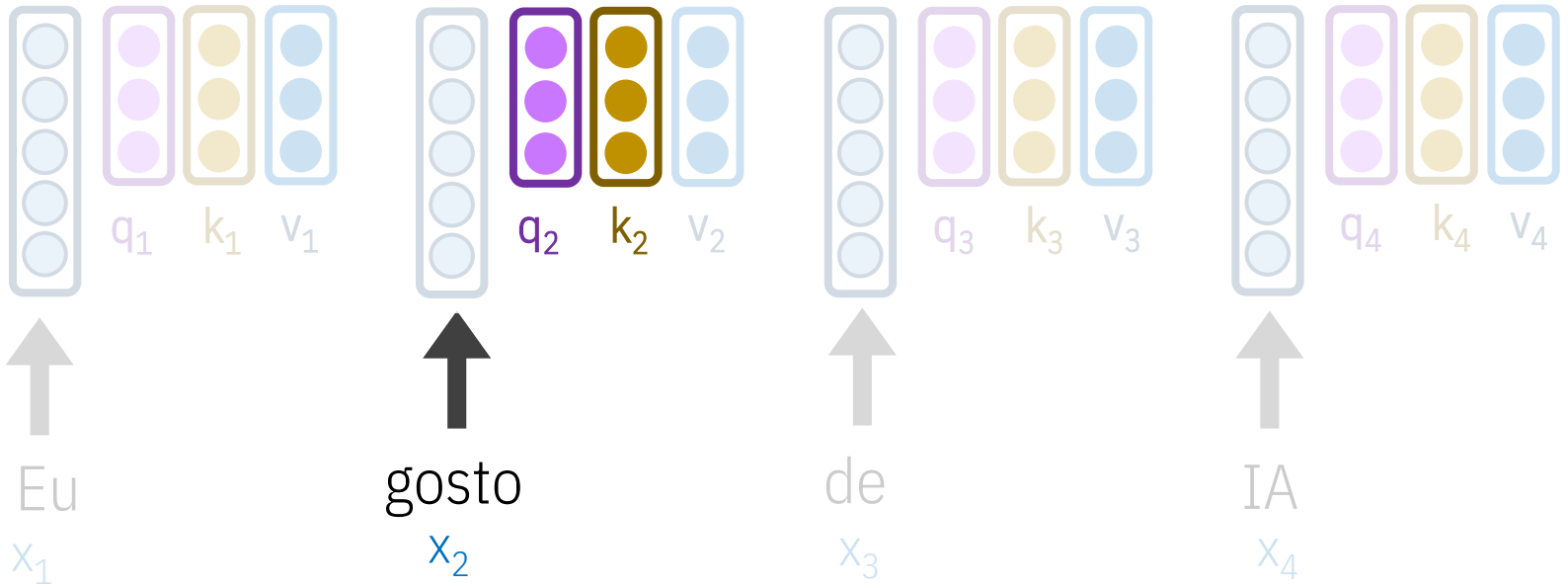


Self-Attention

Passo 2: Para a palavra x_2, vamos calcular os scores s_1, s_2, s_3, s_4, que representam a atenção a prestar a cada uma das respetivos "word" v_i

$s_2 = q_2 \cdot k_2 = 10{,}724$

$s_1 = q_2 \cdot k_1 = 0$



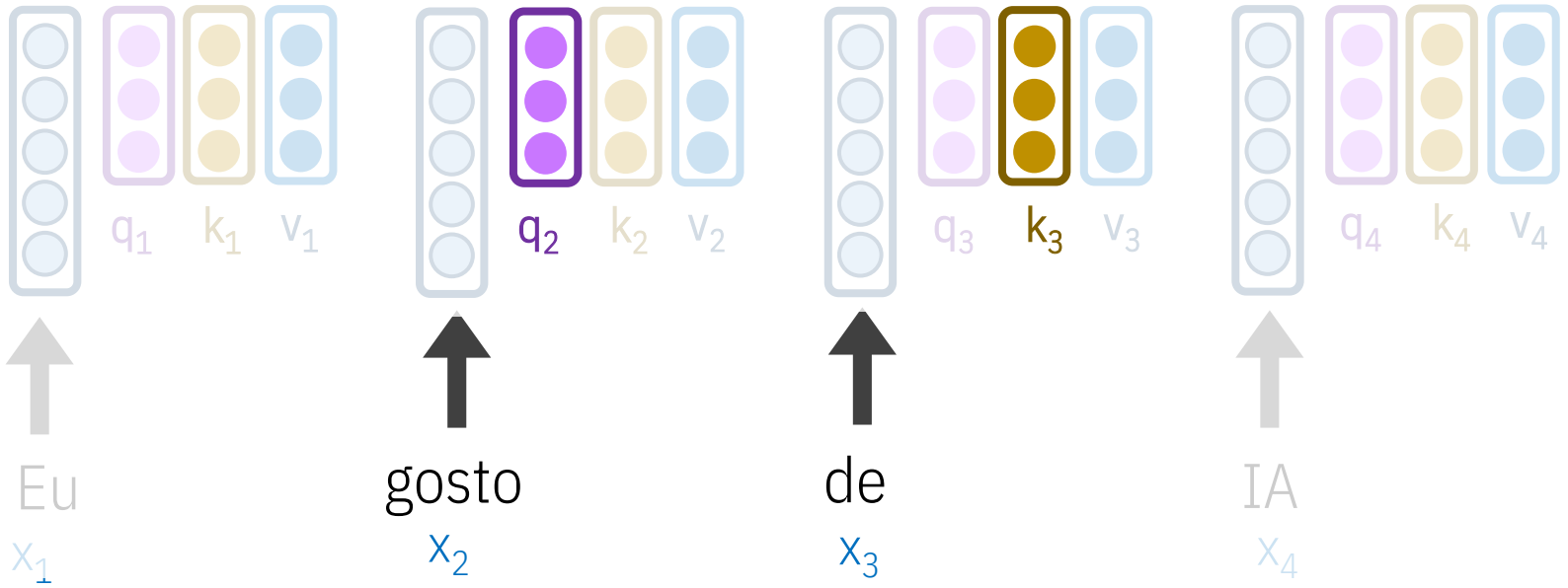
Self-Attention

Passo 2: Para a palavra x_2, vamos calcular os scores s_1, s_2, s_3, s_4, que representam a atenção a prestar a cada uma das respetivos ”word” v_i

$s_3 = q_2 \cdot k_3 = 5{,}068$

$s_2 = q_2 \cdot k_2 = 10{,}724$

$s_1 = q_2 \cdot k_1 = 0$



Self-Attention

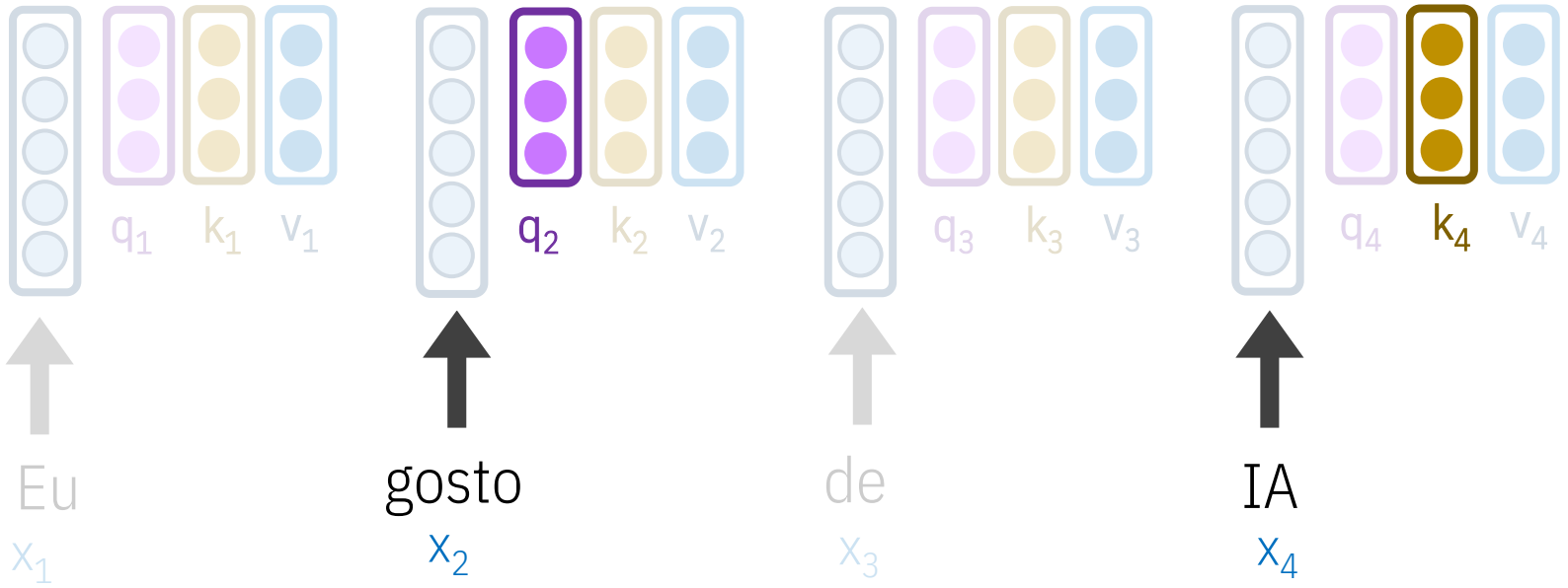
Passo 2: Para a palavra x_2, vamos calcular os scores s_1, s_2, s_3, s_4, que representam a atenção a prestar a cada uma das respetivos ”word” v_i

$s_4 = q_2 \cdot k_4 = 10{,}724$

$s_3 = q_2 \cdot k_3 = 5{,}068$

$s_2 = q_2 \cdot k_2 = 10{,}724$

$s_1 = q_2 \cdot k_1 = 0$



Self-Attention

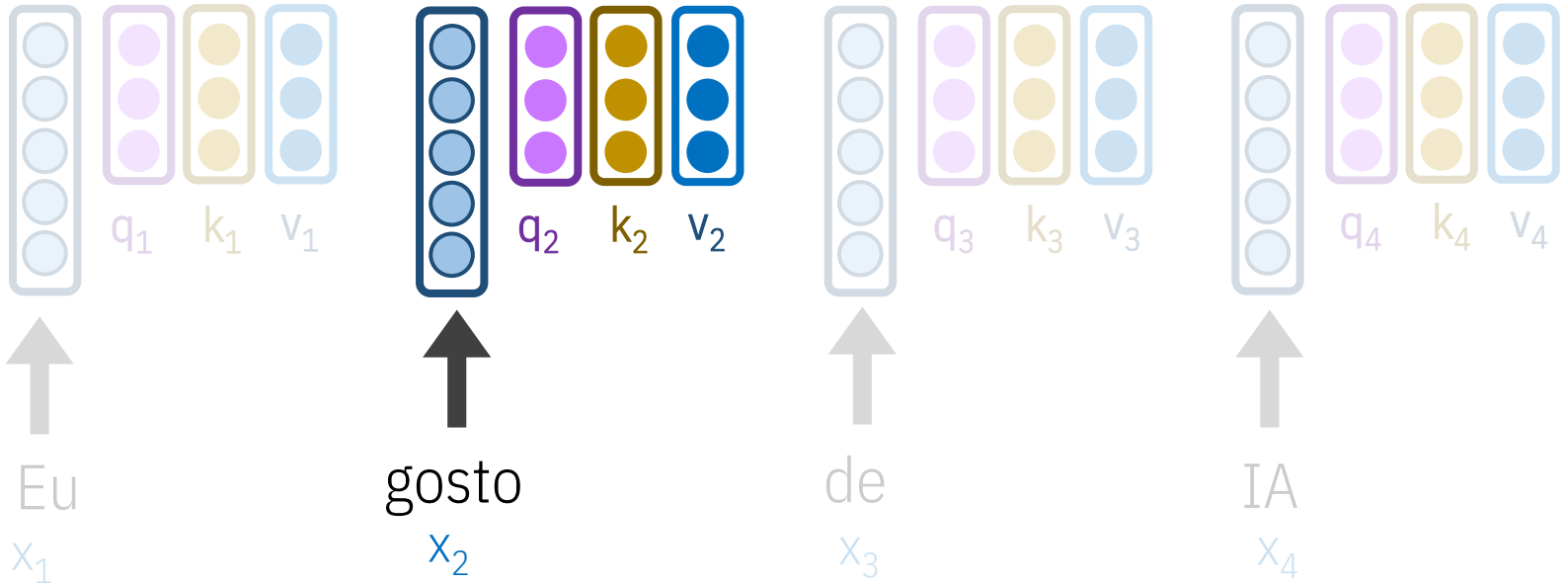
Passo 3: Os nossos scores s_1, s_2, s_3, s_4 não somam 1. Vamos dividir por $\sqrt{len(k_i)}$ e aplicar softmax .

$s_4 = q_2 \cdot k_4 = 10{,}724$ $\quad a_4 = \boldsymbol{\sigma}(s_4/8) = 0{,}363$

$s_3 = q_2 \cdot k_3 = 5{,}068$ $\quad a_3 = \boldsymbol{\sigma}(s_3/8) = 0{,}179$

$s_2 = q_2 \cdot k_2 = 10{,}724$ $\quad a_2 = \boldsymbol{\sigma}(s_2/8) = 0{,}363$

$s_1 = q_2 \cdot k_1 = 0$ $\quad a_1 = \boldsymbol{\sigma}(s_1/8) = 0{,}095$



Self-Attention

Passo 3: Os nossos scores s_1, s_2, s_3, s_4 não somam 1. Vamos dividir por $\sqrt{len(k_i)}$ e aplicar softmax .

$s_4 = q_2 \cdot k_4 = 10{,}724$

$s_3 = q_2 \cdot k_3 = 5{,}068$

$s_2 = q_2 \cdot k_2 = 10{,}724$

$s_1 = q_2 \cdot k_1 = 0$

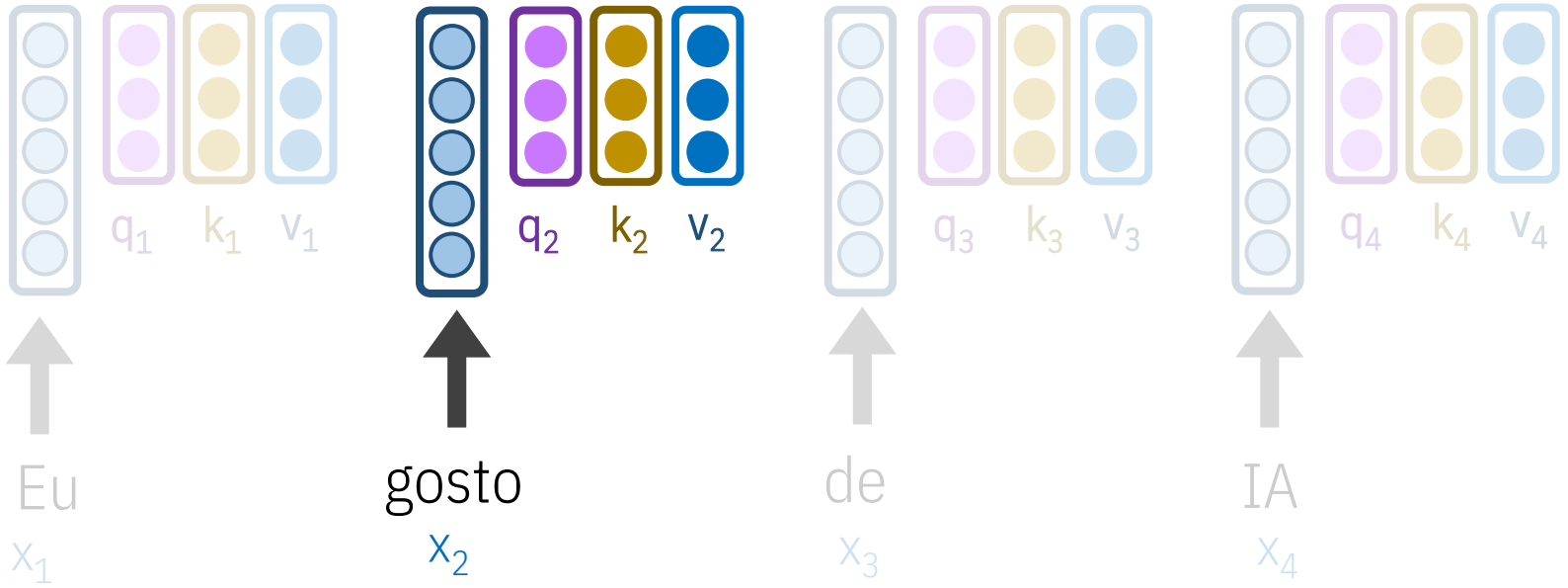
$a_4 = \sigma(s_4/8) = 0{,}363$

$a_3 = \sigma(s_3/8) = 0{,}179$

$a_2 = \sigma(s_2/8) = 0{,}363$

$a_1 = \sigma(s_1/8) = 0{,}095$

Em vez destes valores a_i diretamente pesarem os vetores originais de x_i, eles diretamente pesam os vetores v_i.

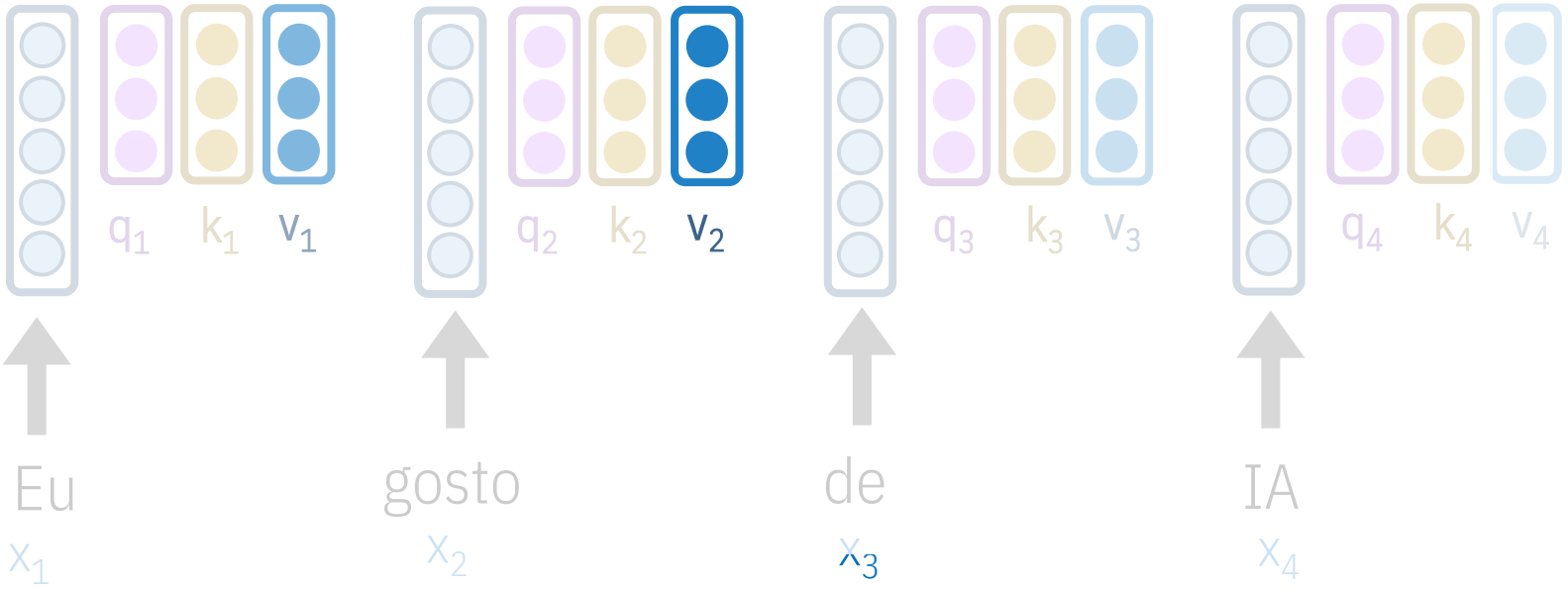


Self-Attention

Passo 4: Vamos pesar os nossos vetores v_i e simplesmente resumi-los!

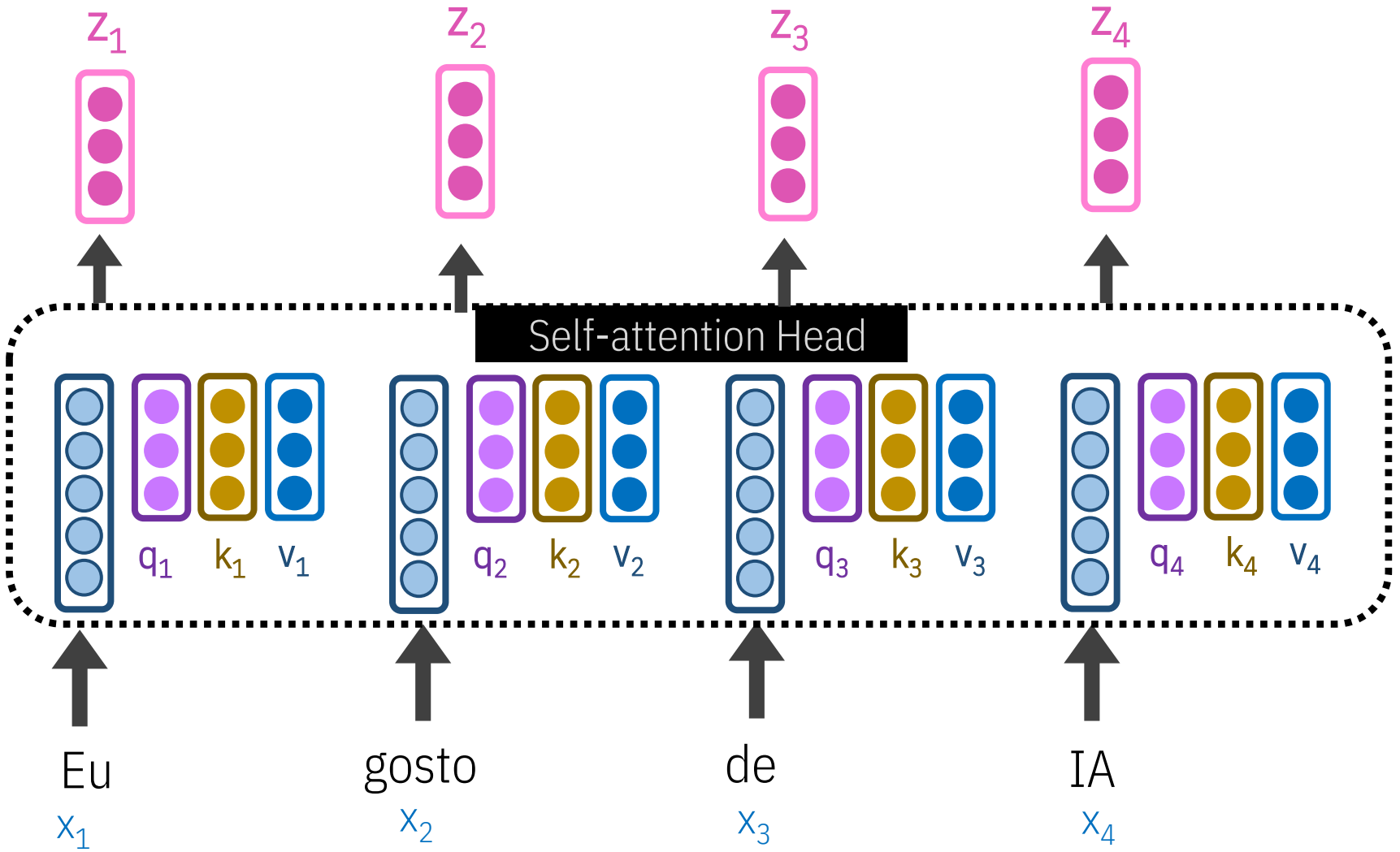


$$z_2 = a_1 \cdot v_1 + a_2 \cdot v_2 + a_3 \cdot v_3 + a_4 \cdot v_4$$
$$= 0{,}095 \cdot v_1 + 0.363 \cdot v_2 + 0.179 \cdot v_3 + 0{,}363 \cdot v_4$$



Self-Attention

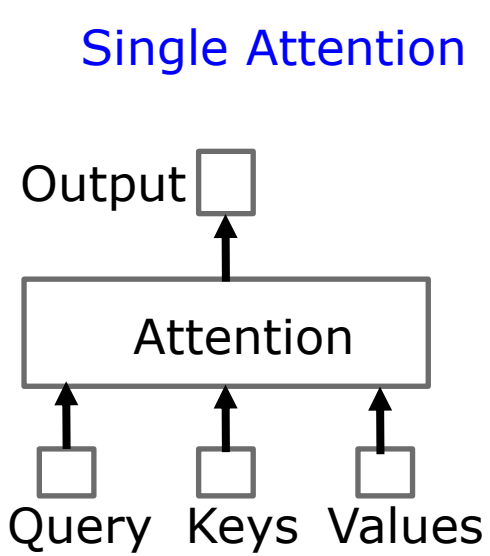
Agora temos novas representações z_i via a self-attention head

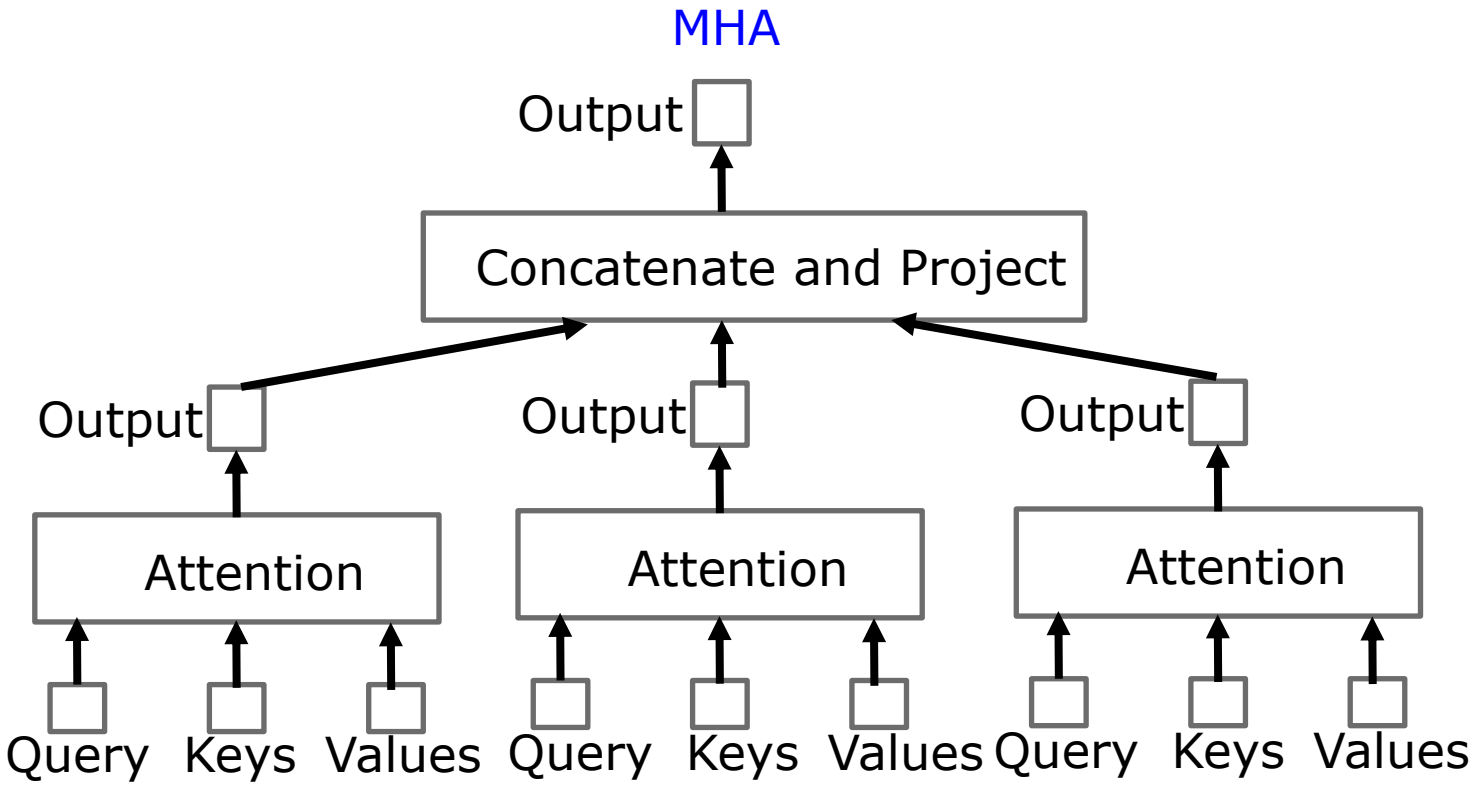


Multi-headed Attention

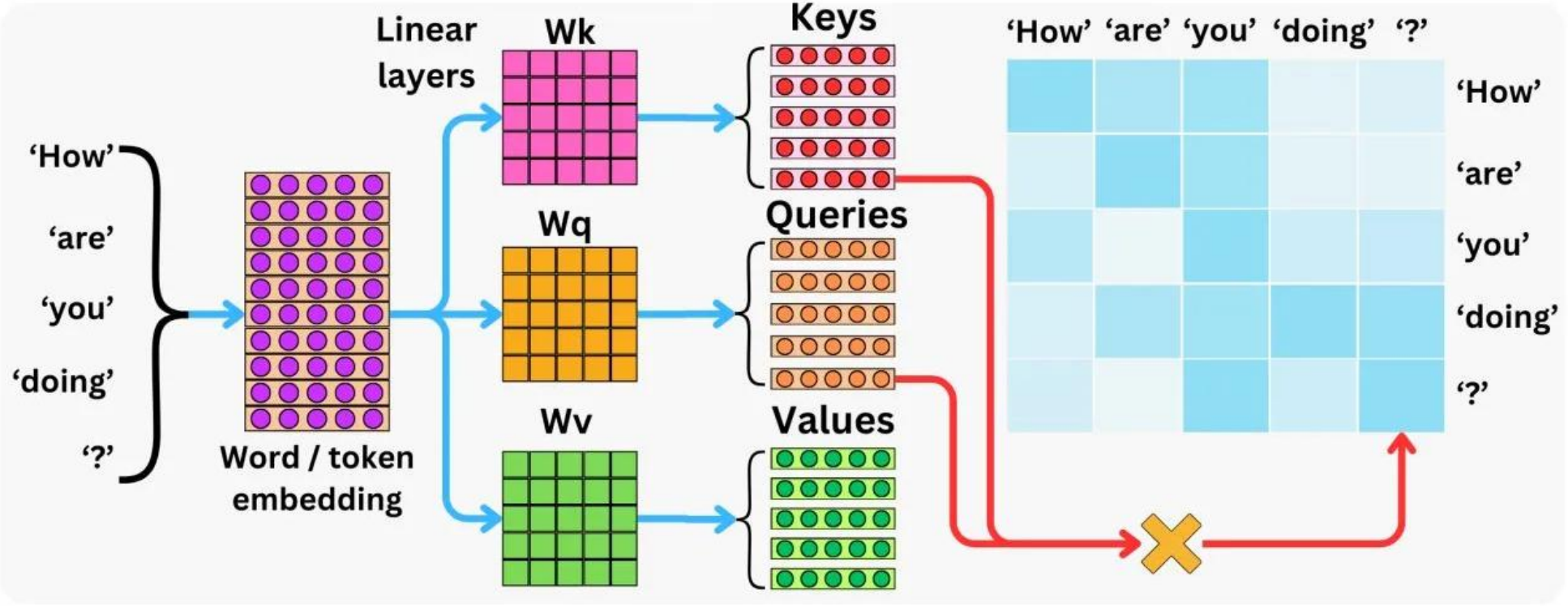
Uma **única função de atenção** pode capturar apenas uma noção de semelhança

Por conseguinte, os transformadores utilizam **multi-head attention** (MHA)



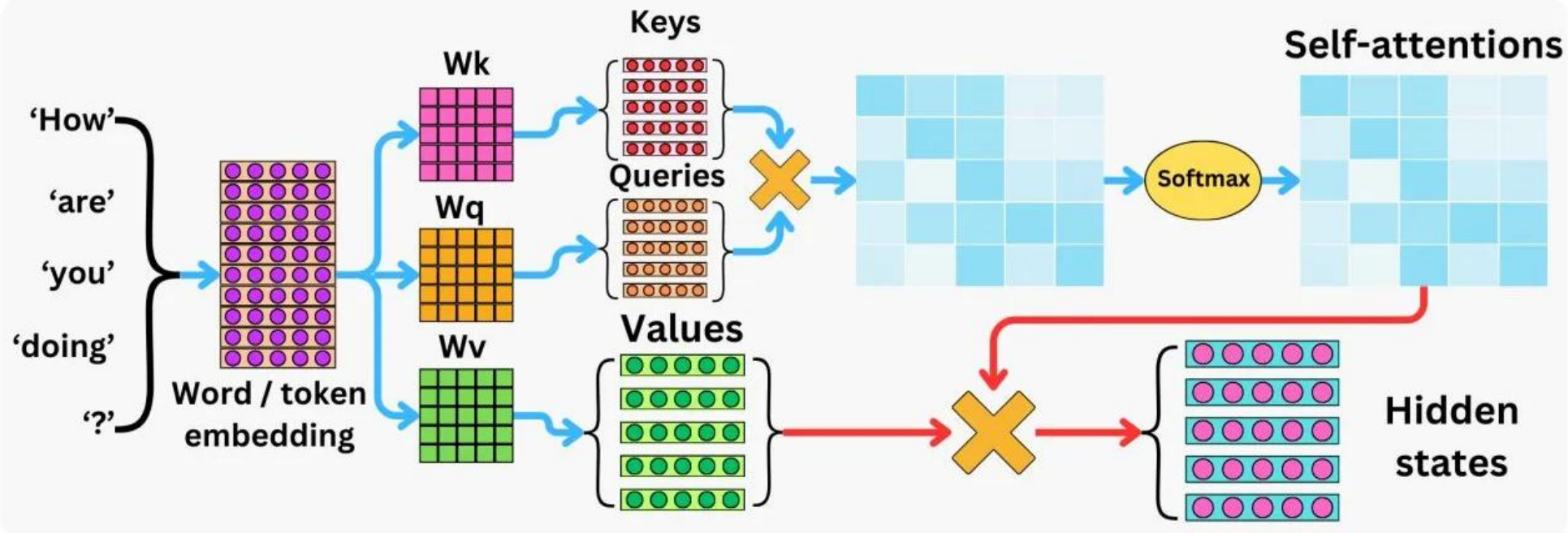


Passo 1: Calcular os valores que capturam a interação token-token.



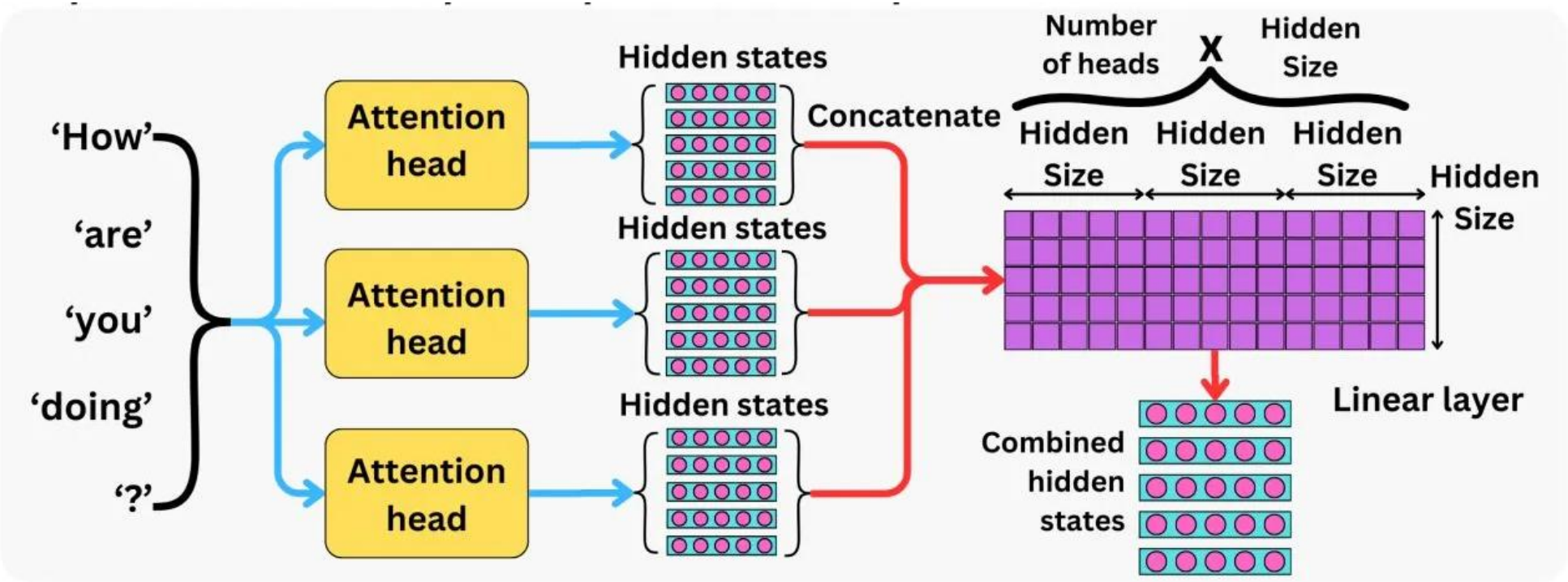
https://poloclub.github.io/transformer-explainer/

Passo 2: Calcular os estados ocultos.



https://poloclub.github.io/transformer-explainer/

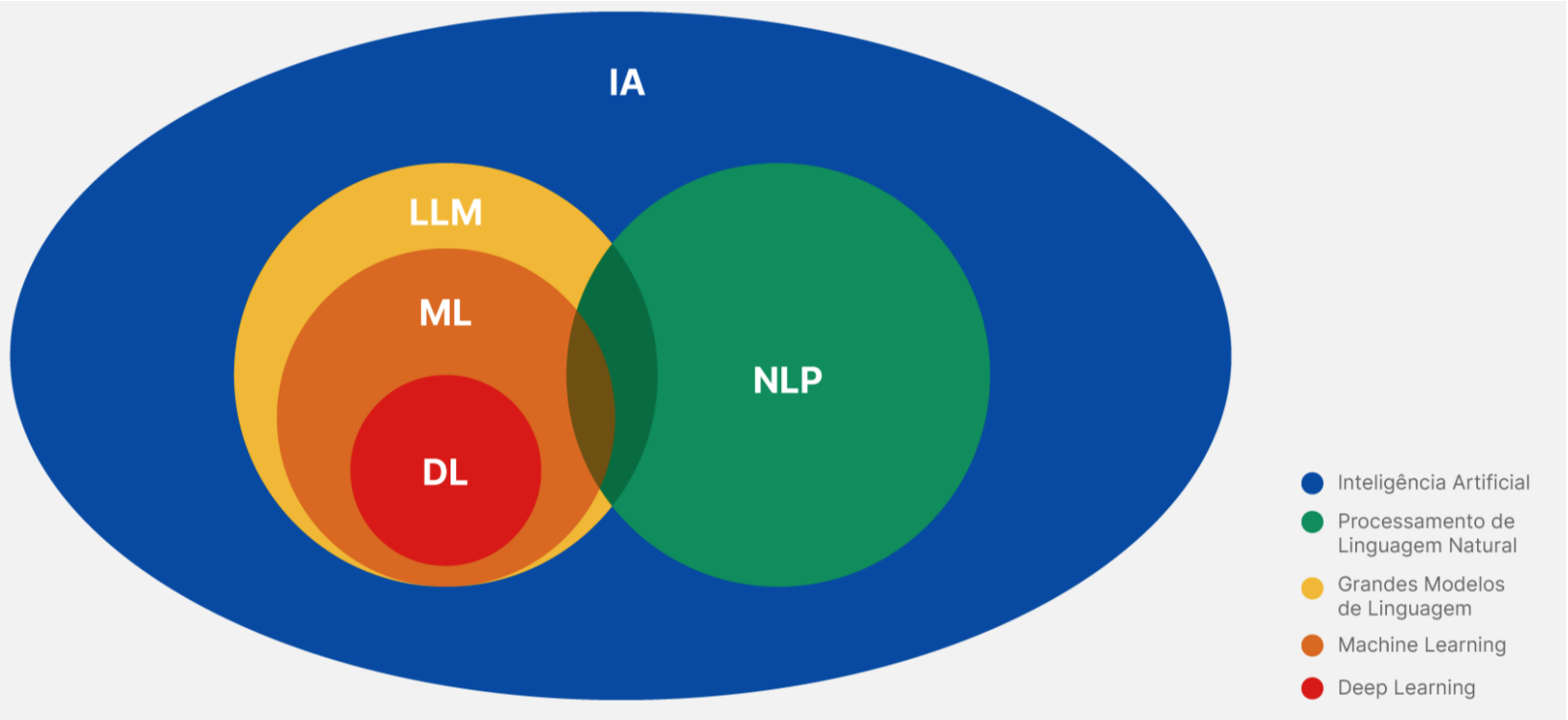
Passo 3: Combinar múltiplas saídas de múltiplas cabeças de Atenção.



https://poloclub.github.io/transformer-explainer/

Aplicações em tarefas de processamento de linguagem natural (PLN).

- Classificação de texto e análise de sentimento
- Sumarização automática
- Tradução e geração de texto
- Pergunta-resposta e assistentes virtuais



Lab

- Lab 03.2 - Transformers